

Convolutional Neural Networks

Dr. Unchalisa Taetragool

Department of Computer Engineering, Faculty of Engineering
King Mongkut's University of Technology Thonburi

Convolutional Neural Network

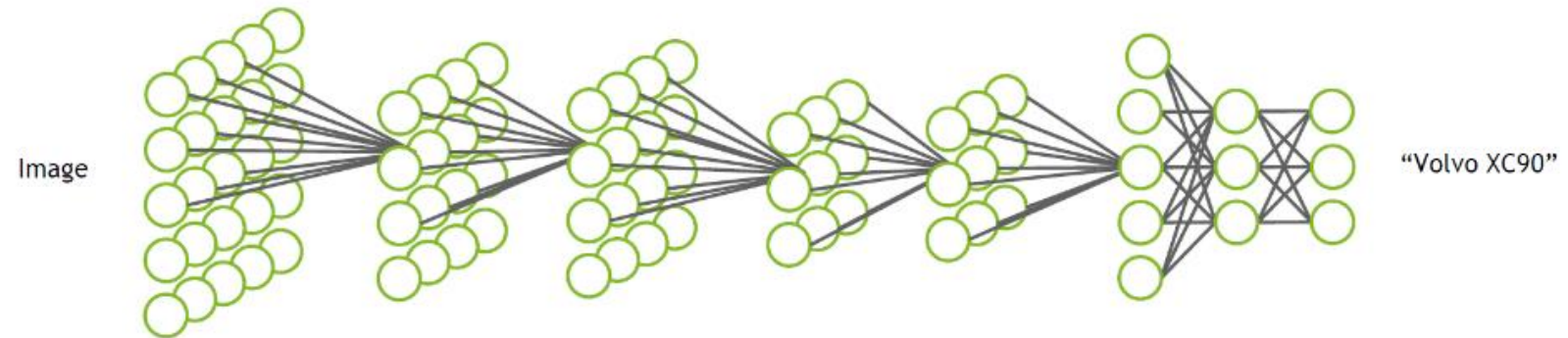
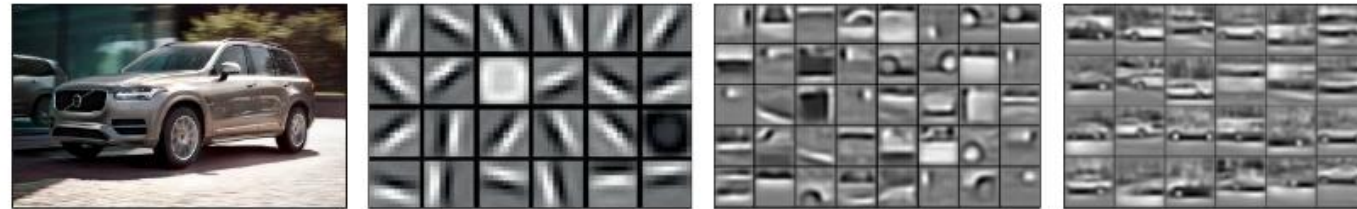


Image source: "Unsupervised Learning of Hierarchical Representations with Convolutional Deep Belief Networks" ICML 2009 & Comm. ACM 2011.
Honglak Lee, Roger Grosse, Rajesh Ranganath, and Andrew Ng.

downside 2

ปัญหา: การที่ AI มีความสามารถในการเรียนรู้ semantic Segmentation : การที่ AI สามารถที่จะเรียนรู้ได้ทั้ง pixel ในภาพ เพื่อที่จะหาว่า pixel นั้นคือส่วนของอะไร

Image Recognition: Simple Example

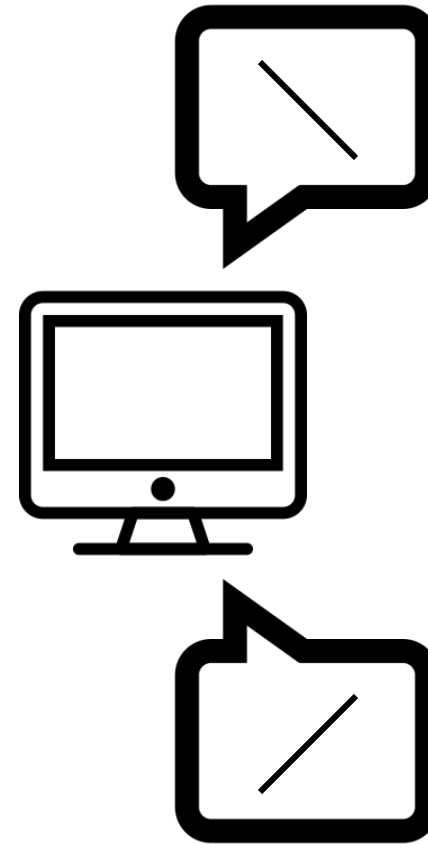
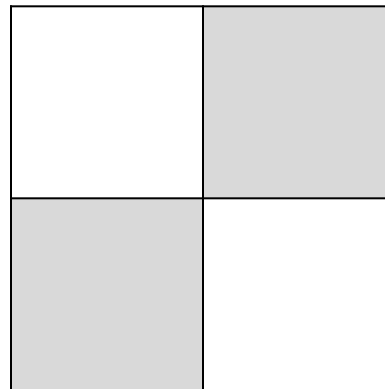
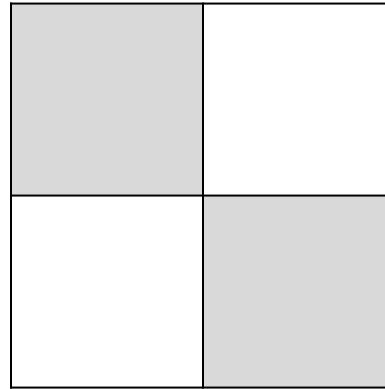
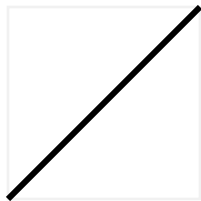
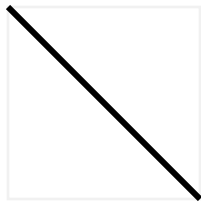
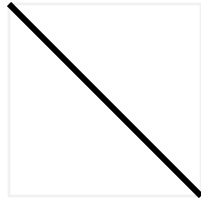
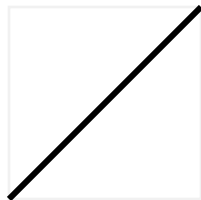


Image Recognition: Simple Example



1	-1
-1	1

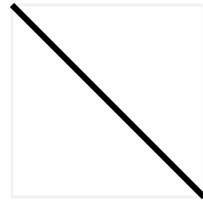
1	-1	-1	1
---	----	----	---



-1	1
1	-1

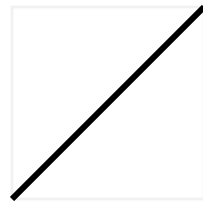
-1	1	1	-1
----	---	---	----

Image Recognition: Simple Example



+	+	+	+
1	-1	-1	1

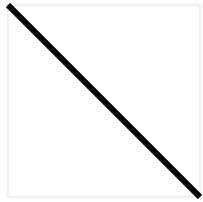
$$+1 \quad -1 \quad -1 \quad +1 = 0$$



+	+	+	+
-1	1	1	-1

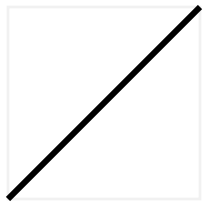
$$-1 \quad 1 \quad 1 \quad -1 = 0$$

Image Recognition: Simple Example



1	-1
-1	1

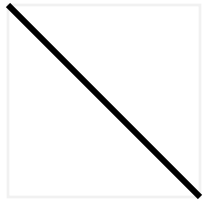
1	-1	-1	1
---	----	----	---



-1	1
1	-1

-1	1	1	-1
----	---	---	----

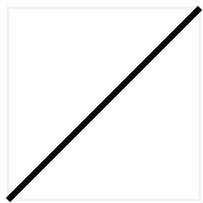
Image Recognition: Simple Example



⁺ 1	⁻ -1
⁻ -1	⁺ 1

⁺	⁻	⁻	⁺
1	-1	-1	1

$$+1 +1 +1 +1 = 4$$



⁺ -1	⁻ 1
⁻ 1	⁺ -1

⁺	⁻	⁻	⁺
-1	1	1	-1

$$-1 -1 -1 -1 = -4$$

Image Recognition: Simple Example

+	-
-	+

If positive, “\”

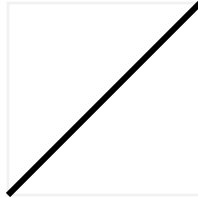
If negative, “/”

Image Recognition: Simple Example

1	1
-1	1

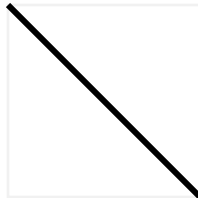
-1	-1
1	-1

+	-	-	+
1	1	-1	1



$+1 -1 +1 +1 = 2$

+	-	-	+
-1	-1	1	-1



$-1 +1 -1 -1 = -2$

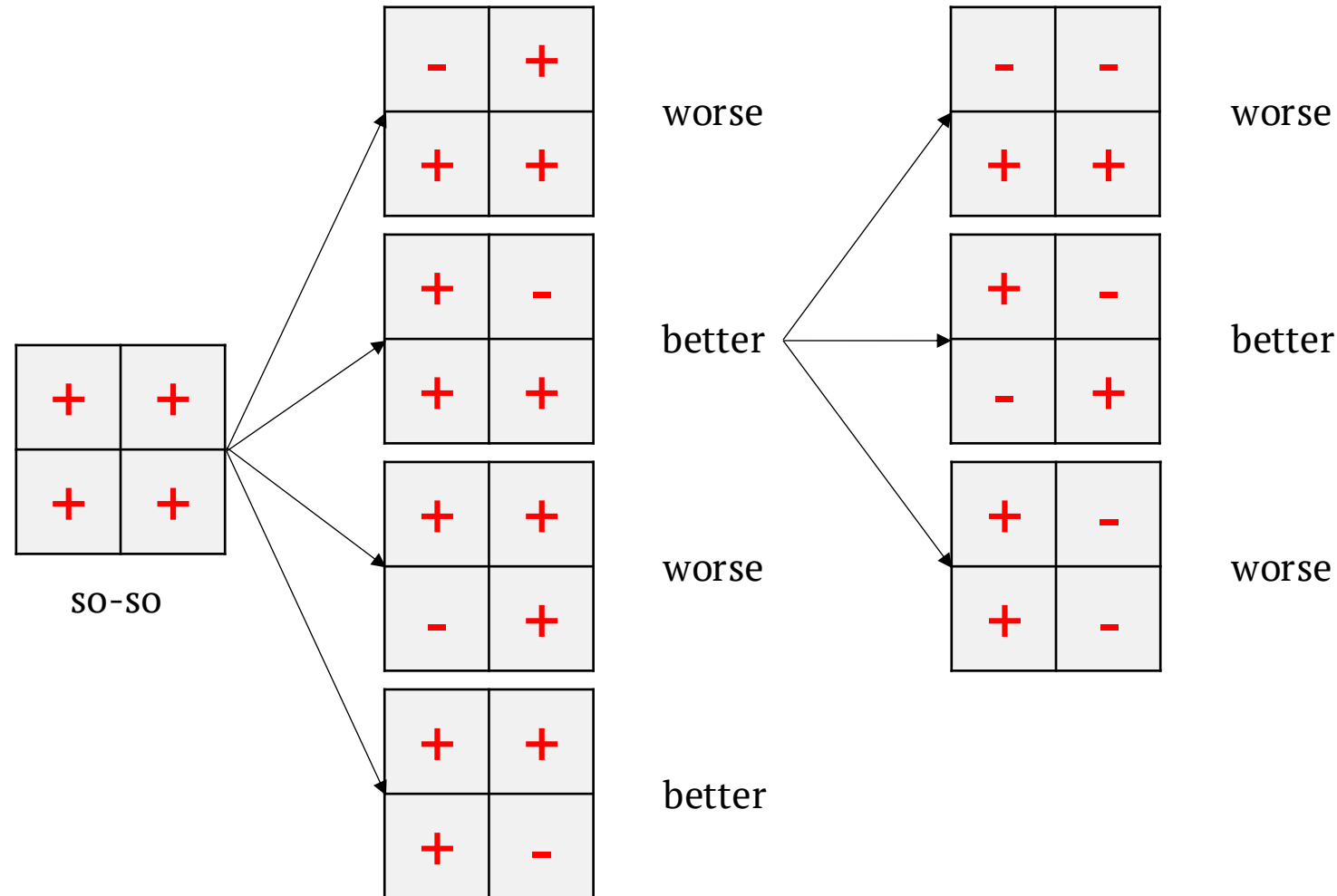
How can a computer find the filters?

<table><tr><td>+</td><td>+</td></tr><tr><td>+</td><td>+</td></tr></table>	+	+	+	+	<table><tr><td>-</td><td>+</td></tr><tr><td>+</td><td>+</td></tr></table>	-	+	+	+	<table><tr><td>+</td><td>-</td></tr><tr><td>+</td><td>+</td></tr></table>	+	-	+	+	<table><tr><td>+</td><td>+</td></tr><tr><td>-</td><td>+</td></tr></table>	+	+	-	+
+	+																		
+	+																		
-	+																		
+	+																		
+	-																		
+	+																		
+	+																		
-	+																		
<table><tr><td>+</td><td>+</td></tr><tr><td>+</td><td>-</td></tr></table>	+	+	+	-	<table><tr><td>-</td><td>-</td></tr><tr><td>+</td><td>+</td></tr></table>	-	-	+	+	<table><tr><td>-</td><td>+</td></tr><tr><td>-</td><td>+</td></tr></table>	-	+	-	+	<table><tr><td>+</td><td>+</td></tr><tr><td>-</td><td>-</td></tr></table>	+	+	-	-
+	+																		
+	-																		
-	-																		
+	+																		
-	+																		
-	+																		
+	+																		
-	-																		

and more ...

Find the best filters from the 16 choices

How can a computer find the filters?



How can a computer find the filters?

0.5	1.2
0.7	1.0

0.6	0.9
0.5	1.1

0.9	-0.7
-0.6	1.1

and so on...

Gradient Descent

0.5	1.2
0.7	1.0

0.6	0.9
0.5	1.1

0.9	-0.7
-0.6	1.1

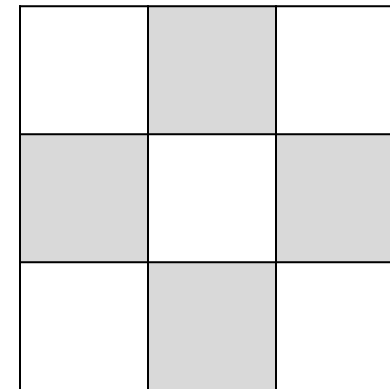
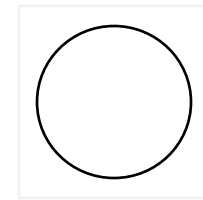
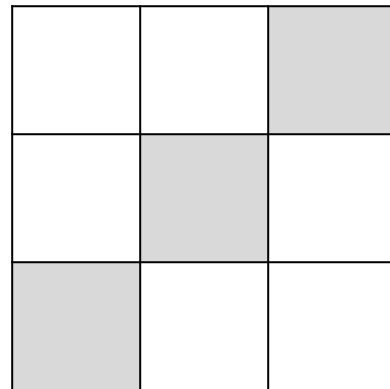
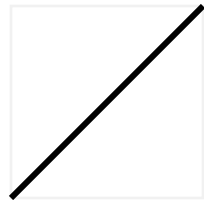
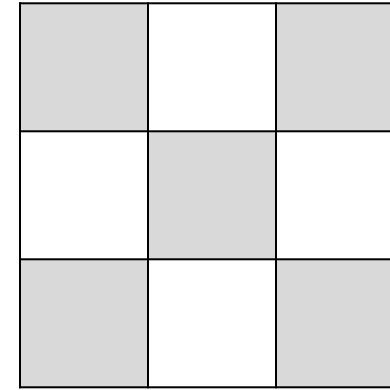
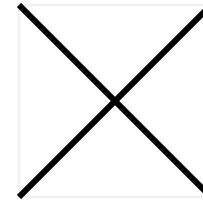
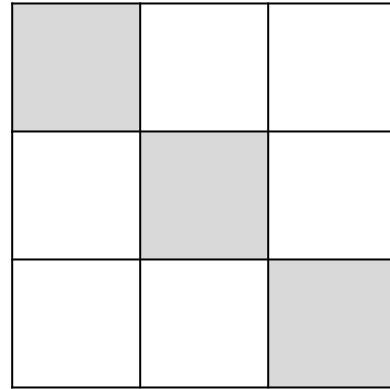
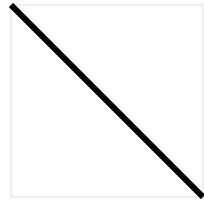
1	-1
-1	1

Derivatives!

Lots of errors

Few errors

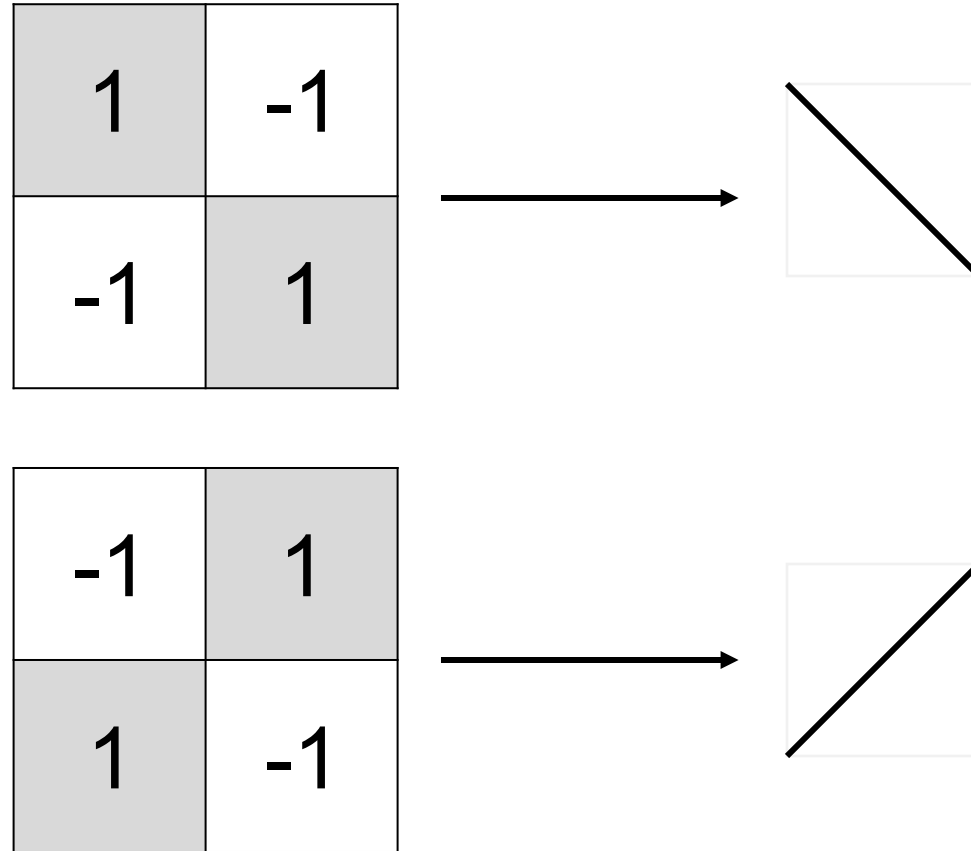
Image Recognition: More Complex Example



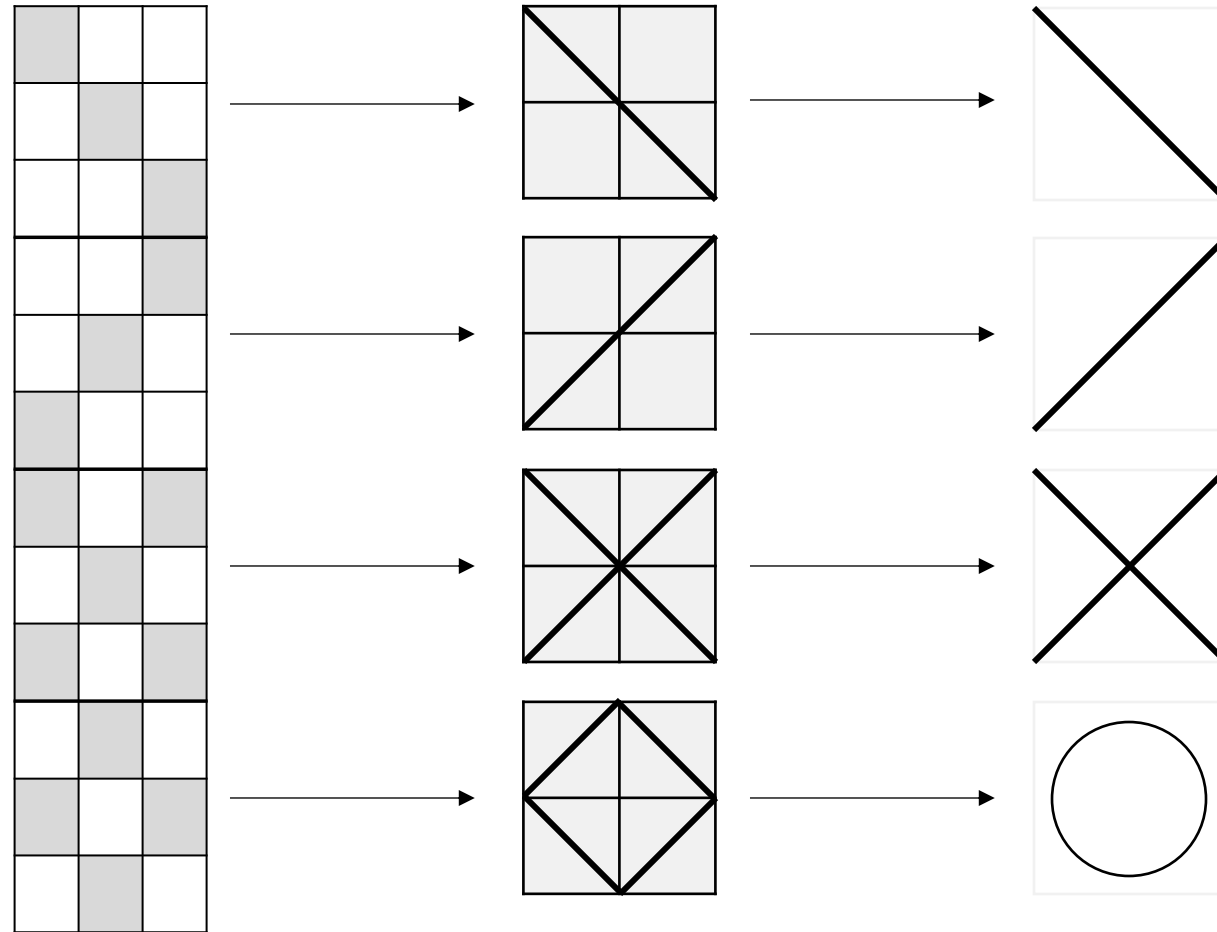
How can a computer find the filters?

⁺ 1	⁻ -1	⁺ 1
⁻ -1	⁺ 1	⁻ -1
⁺ 1	⁻ -1	⁺ 1

Previous Knowledge



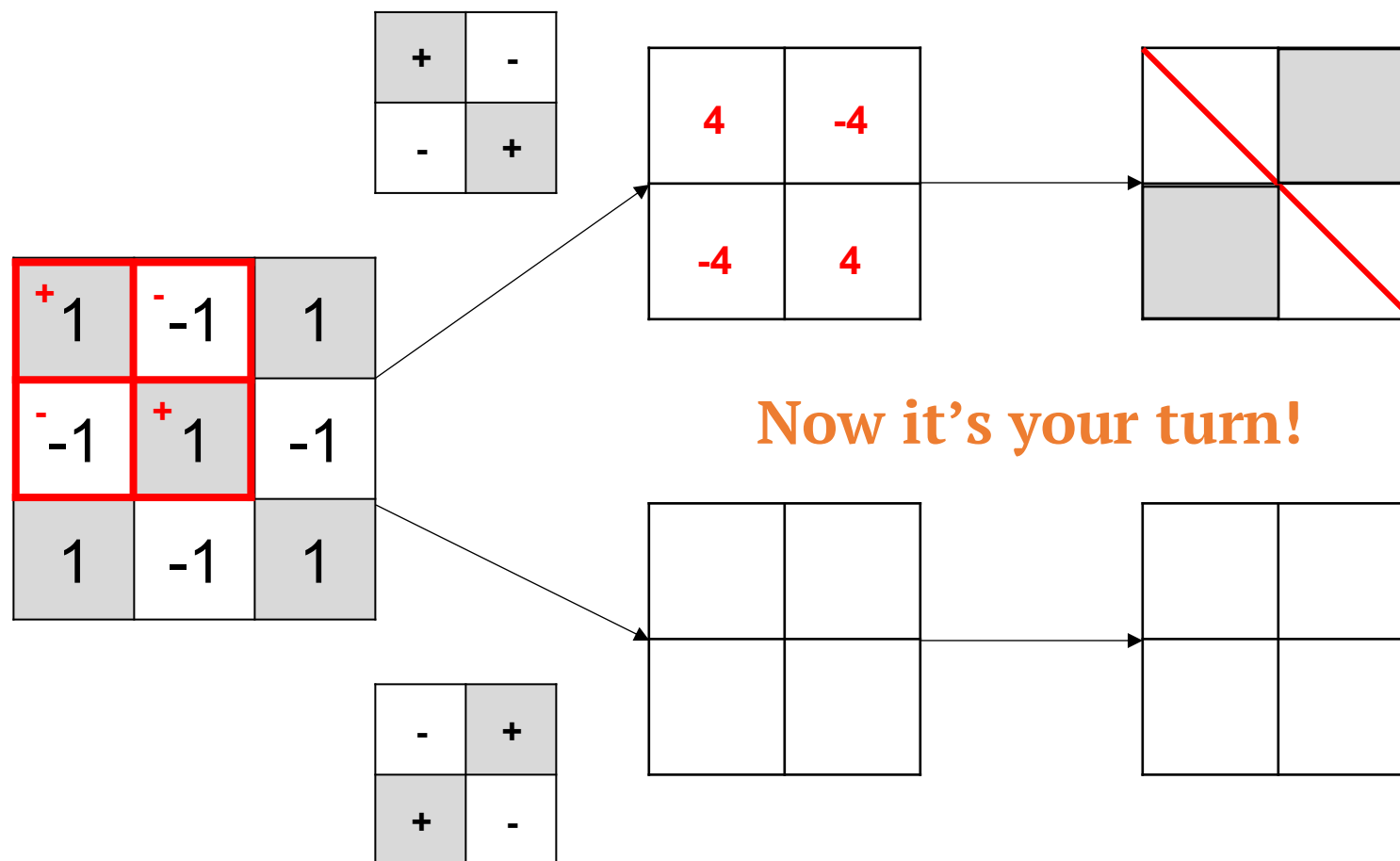
Convolutional Neural Network



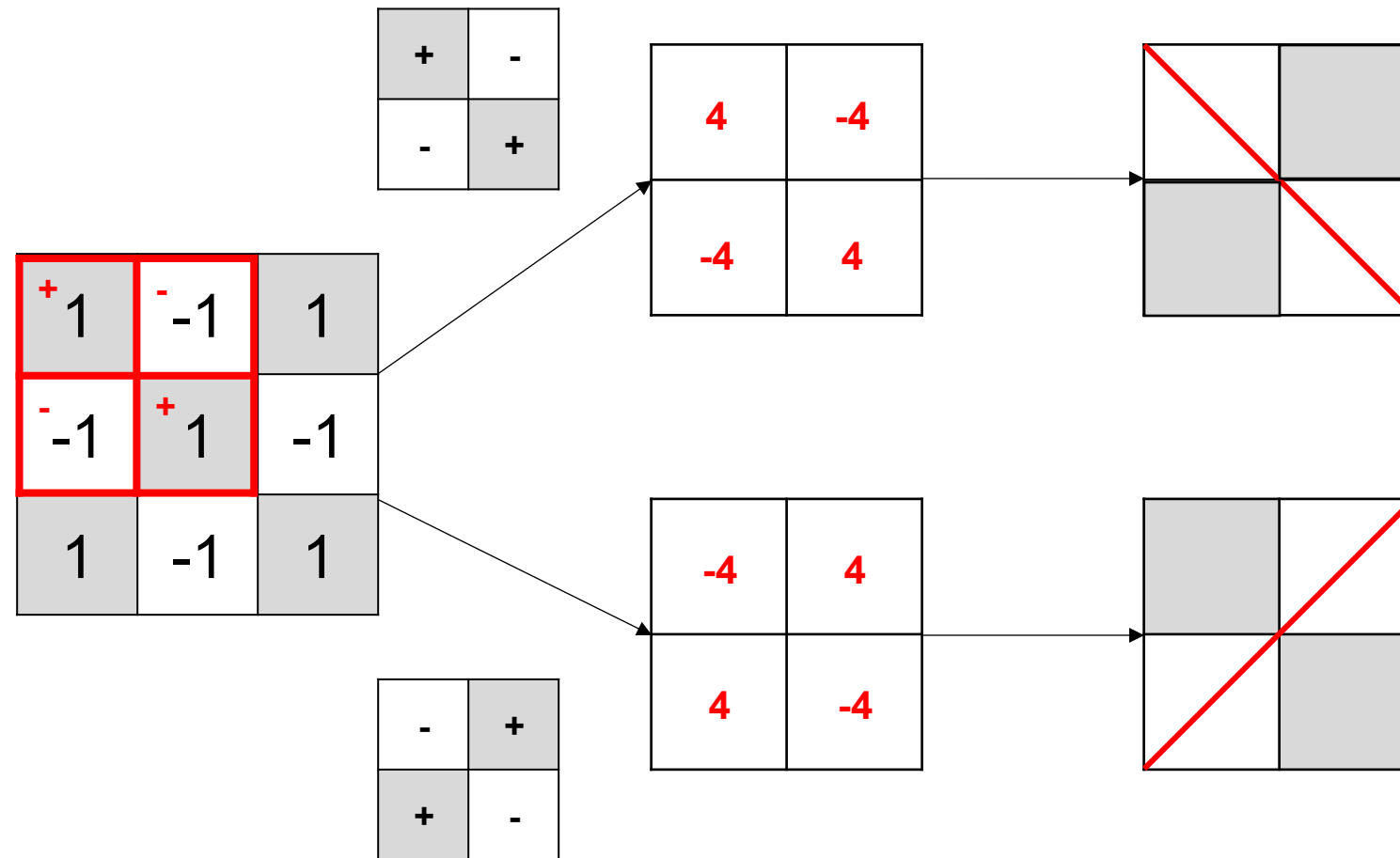
Convolutional Layer
Pooling Layer

Fully Connected Layer

Convolutional Neural Network



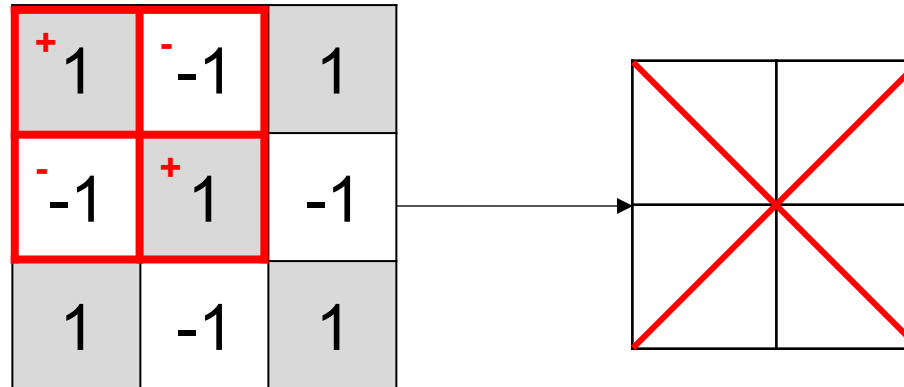
Convolutional Neural Network



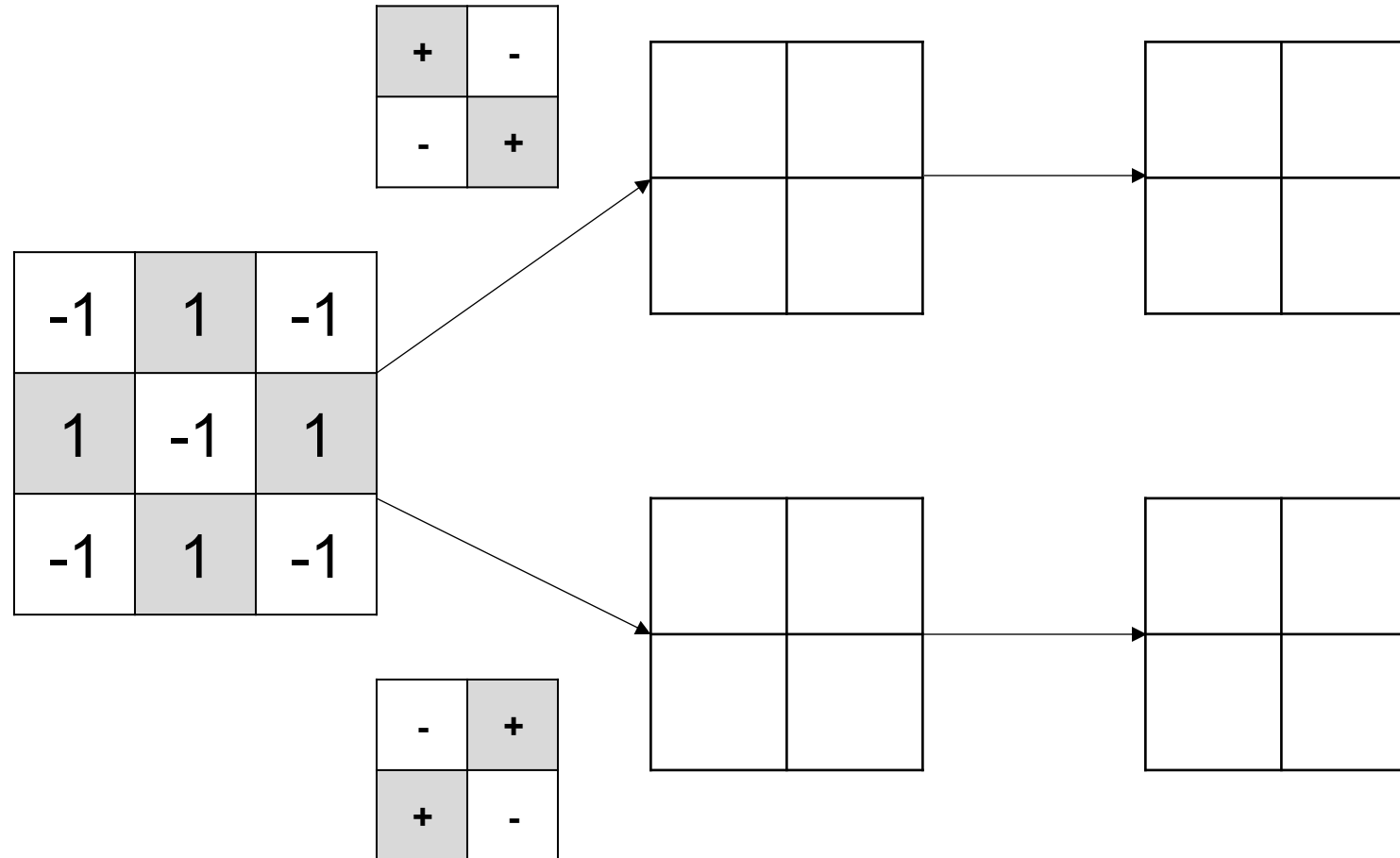
Convolutional Layer

Pooling Layer

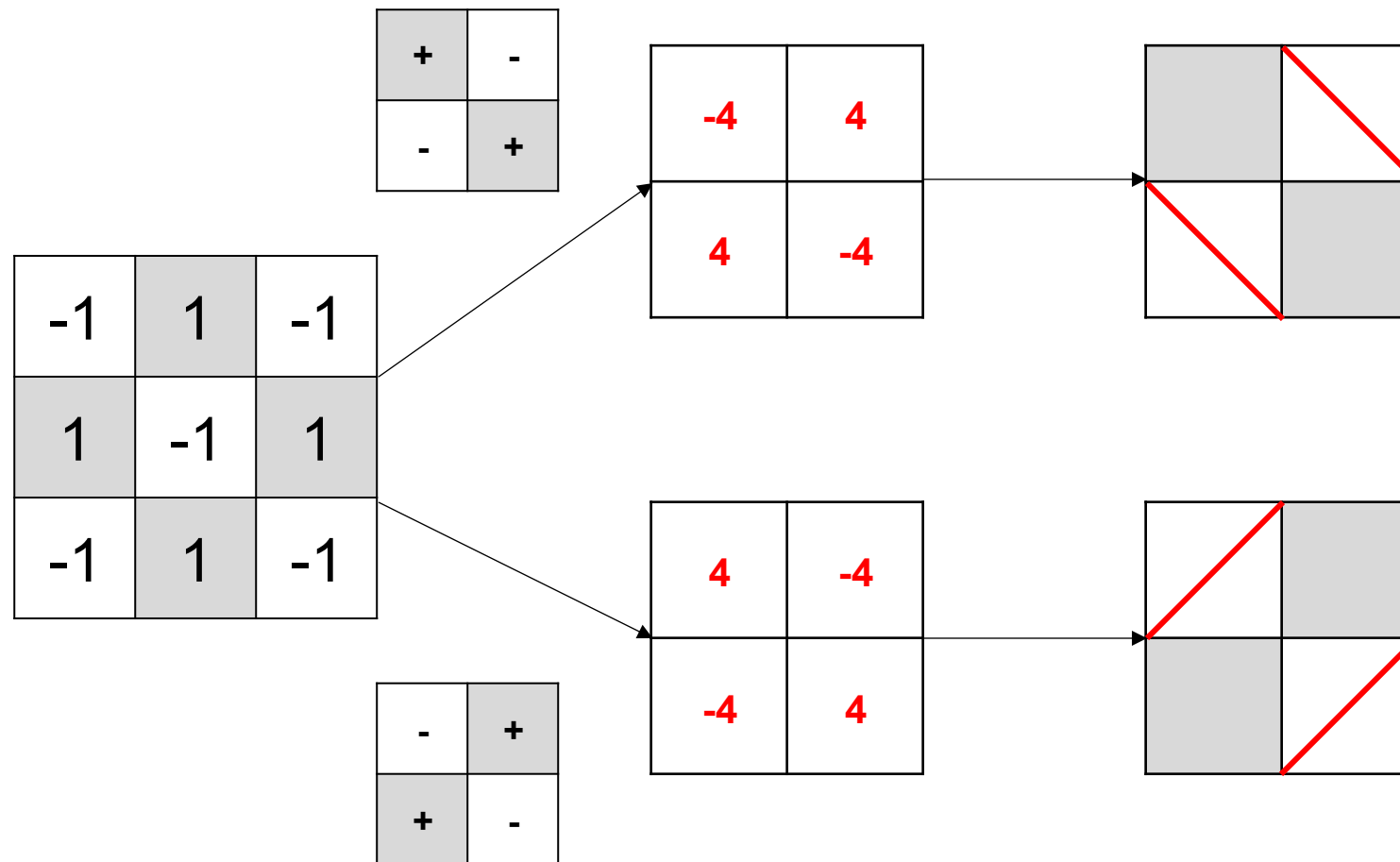
Convolutional Neural Network



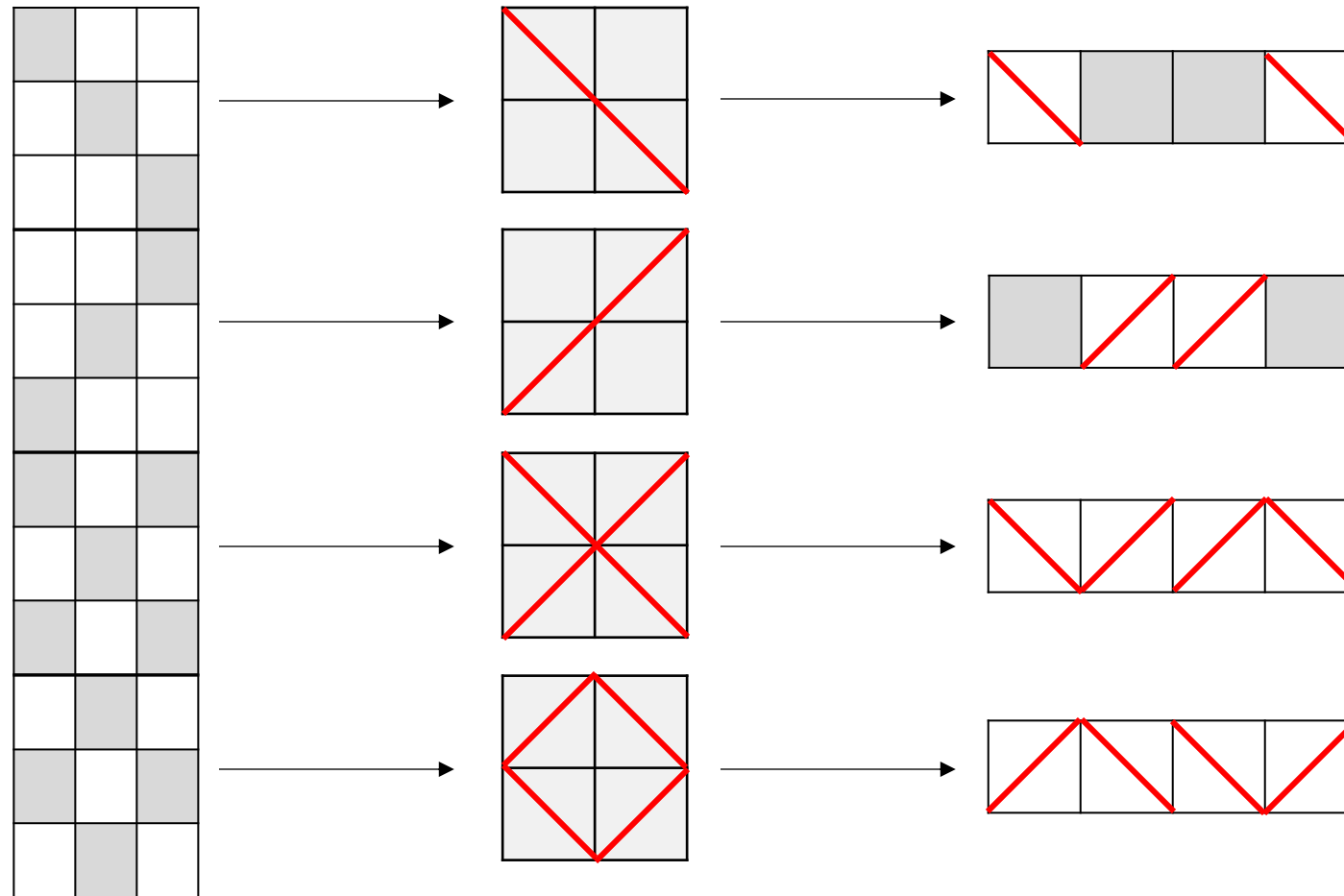
Convolutional Neural Network



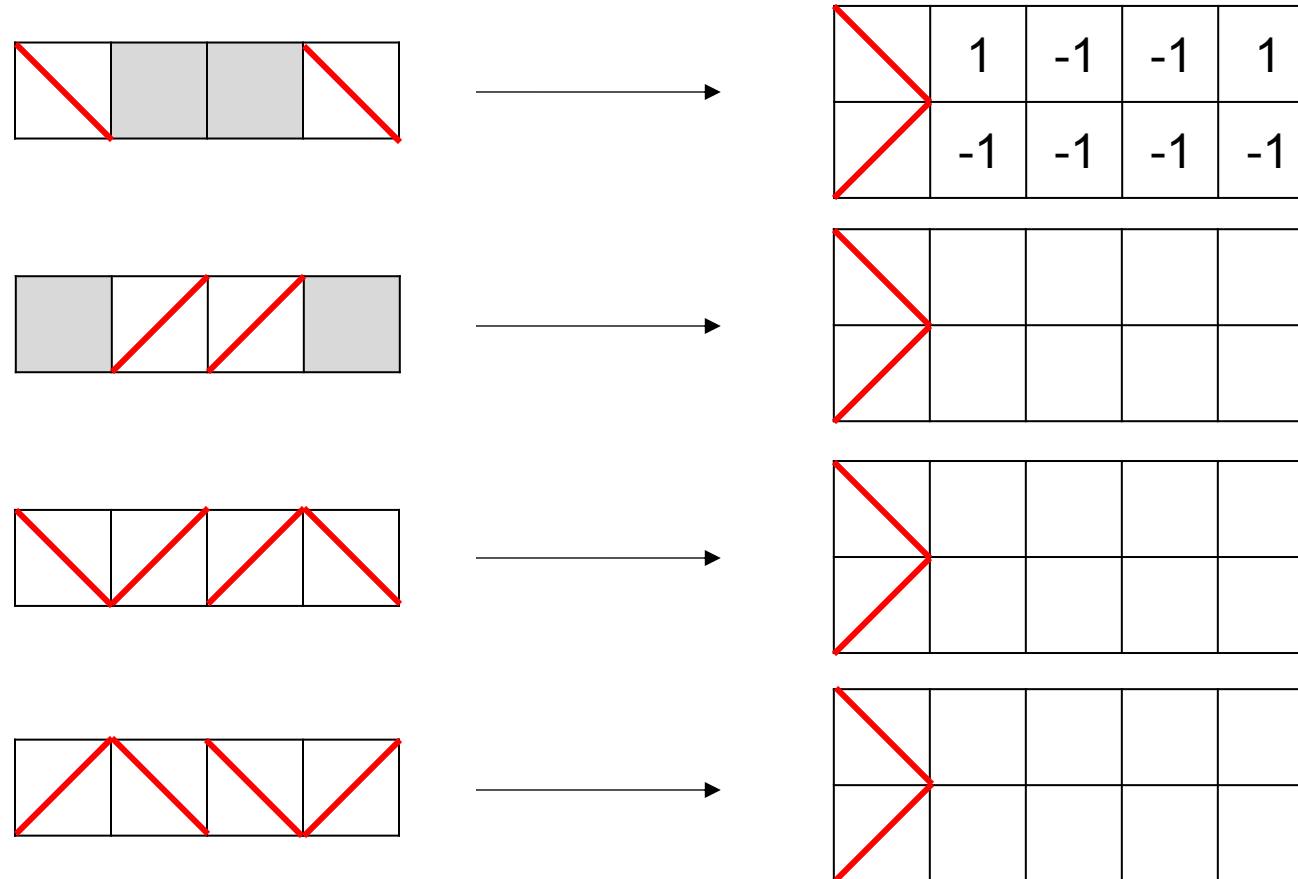
Convolutional Neural Network



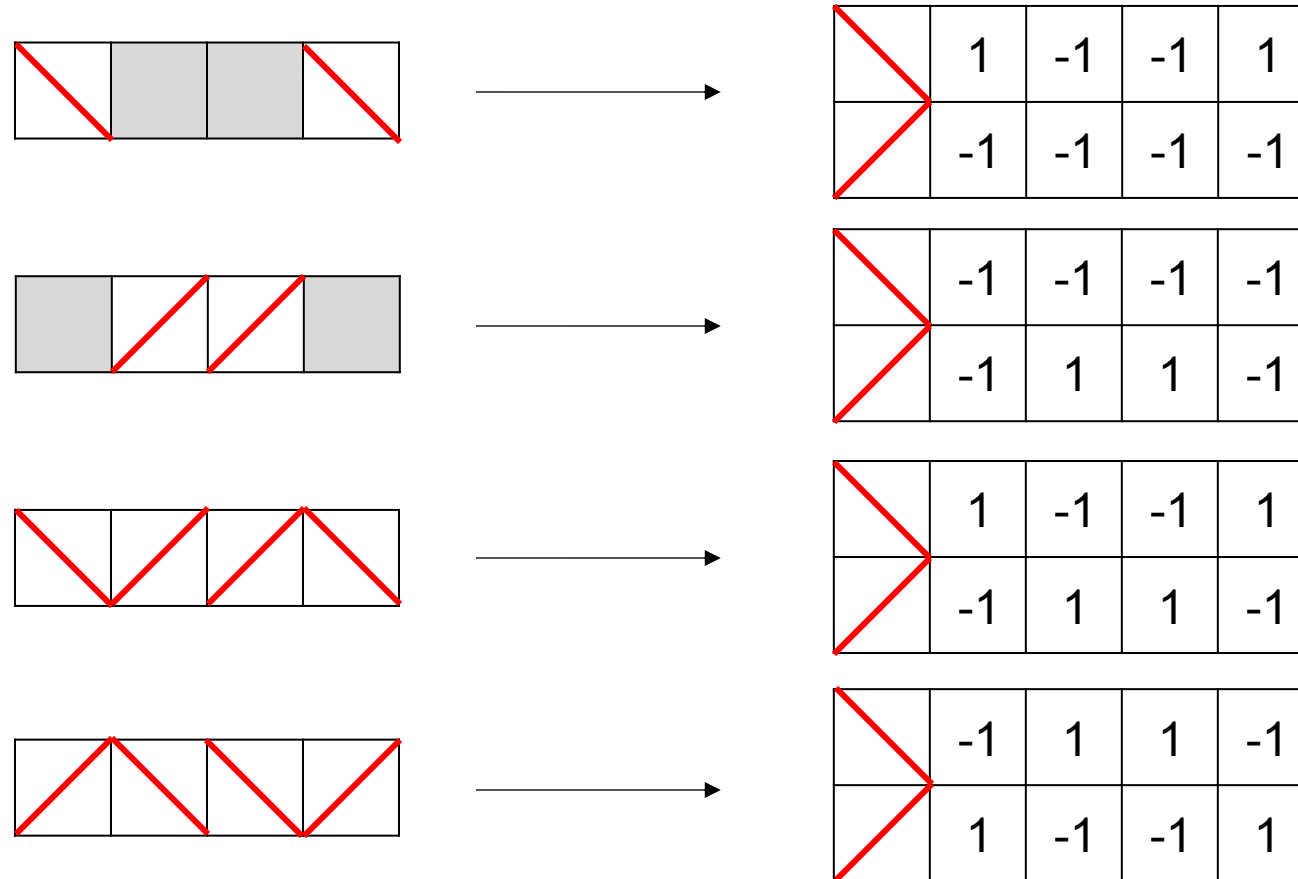
Convolutional Neural Network



Convolutional Neural Network

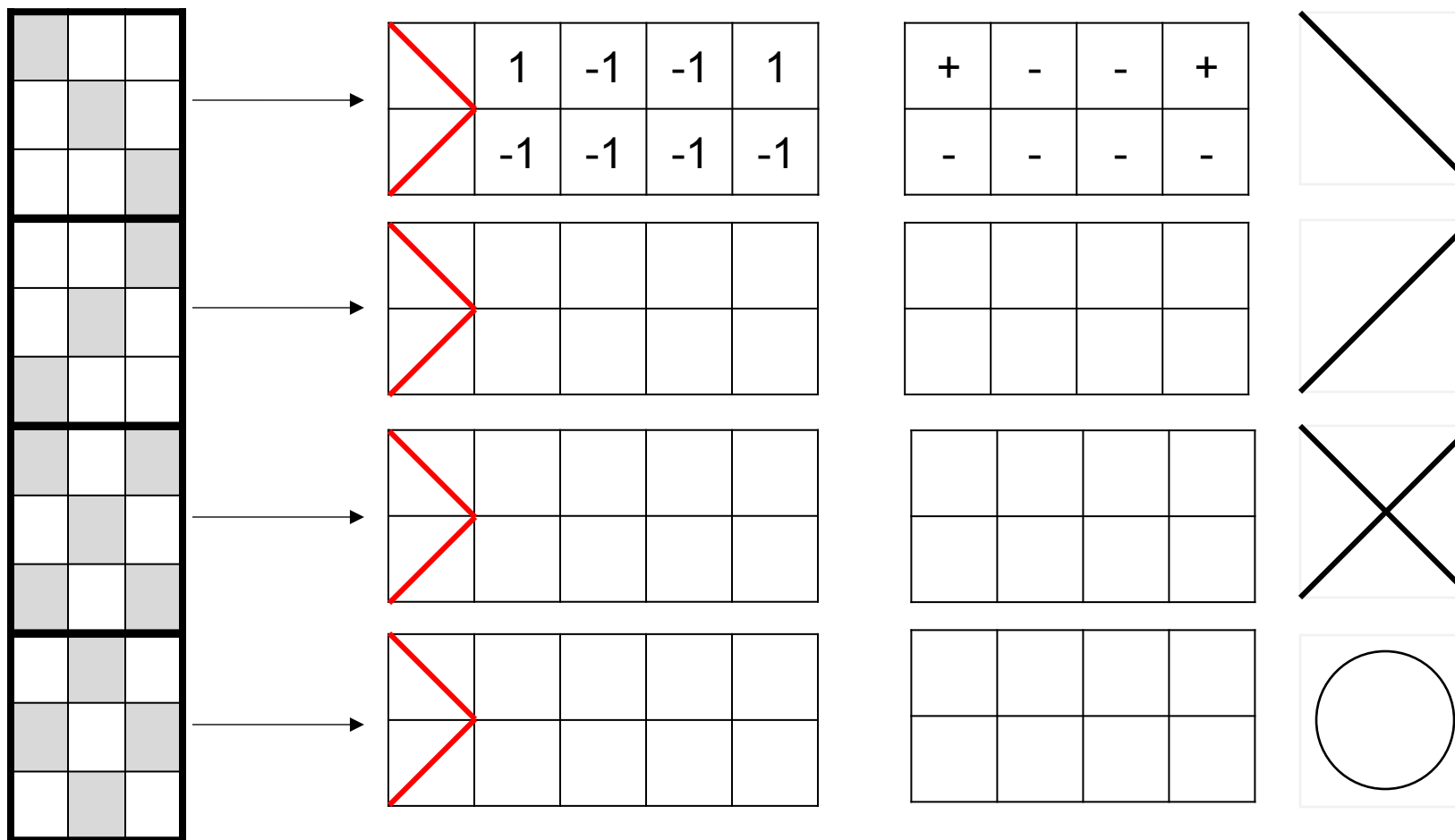


Convolutional Neural Network



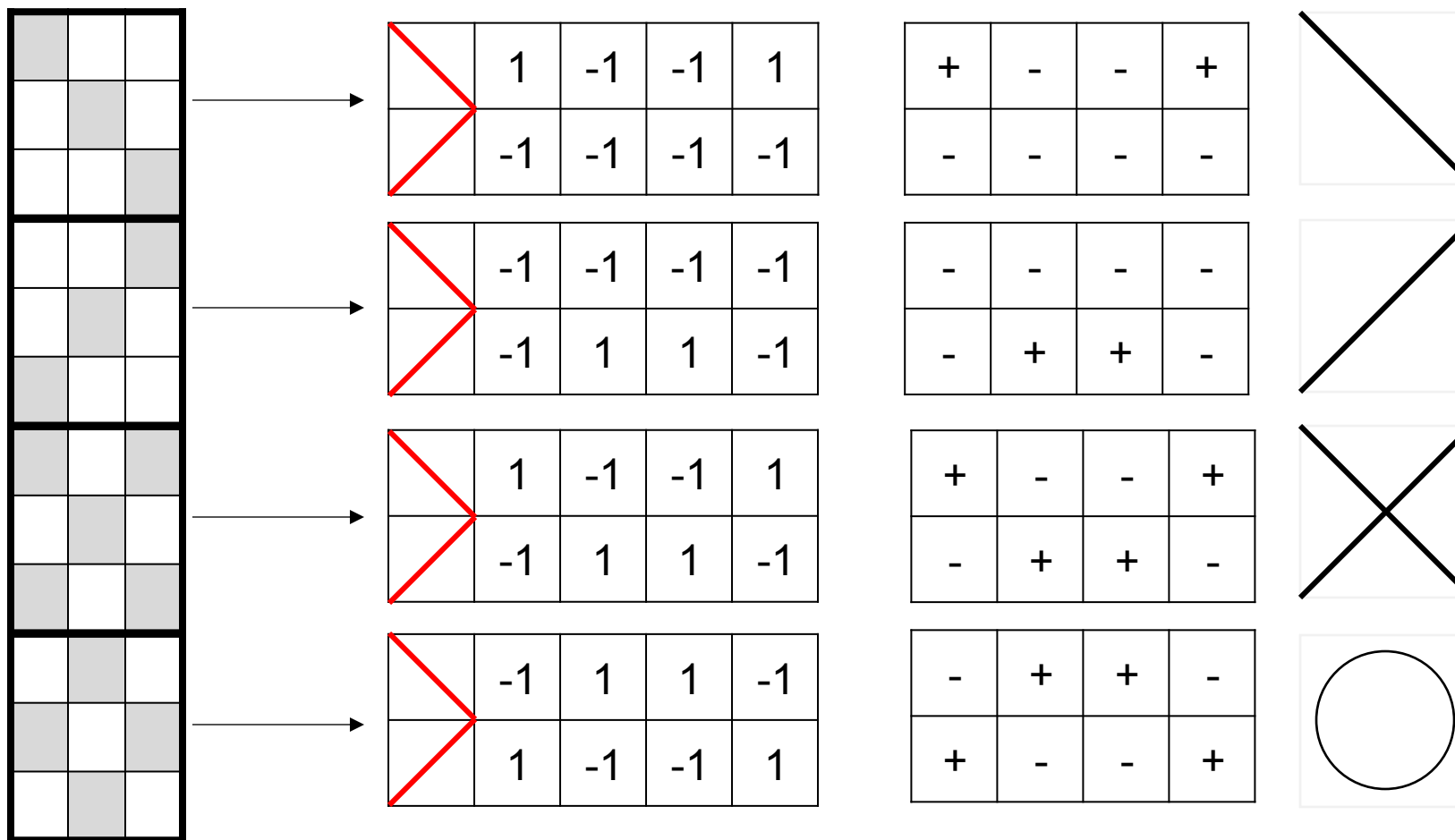
Convolutional Neural Network

Filters

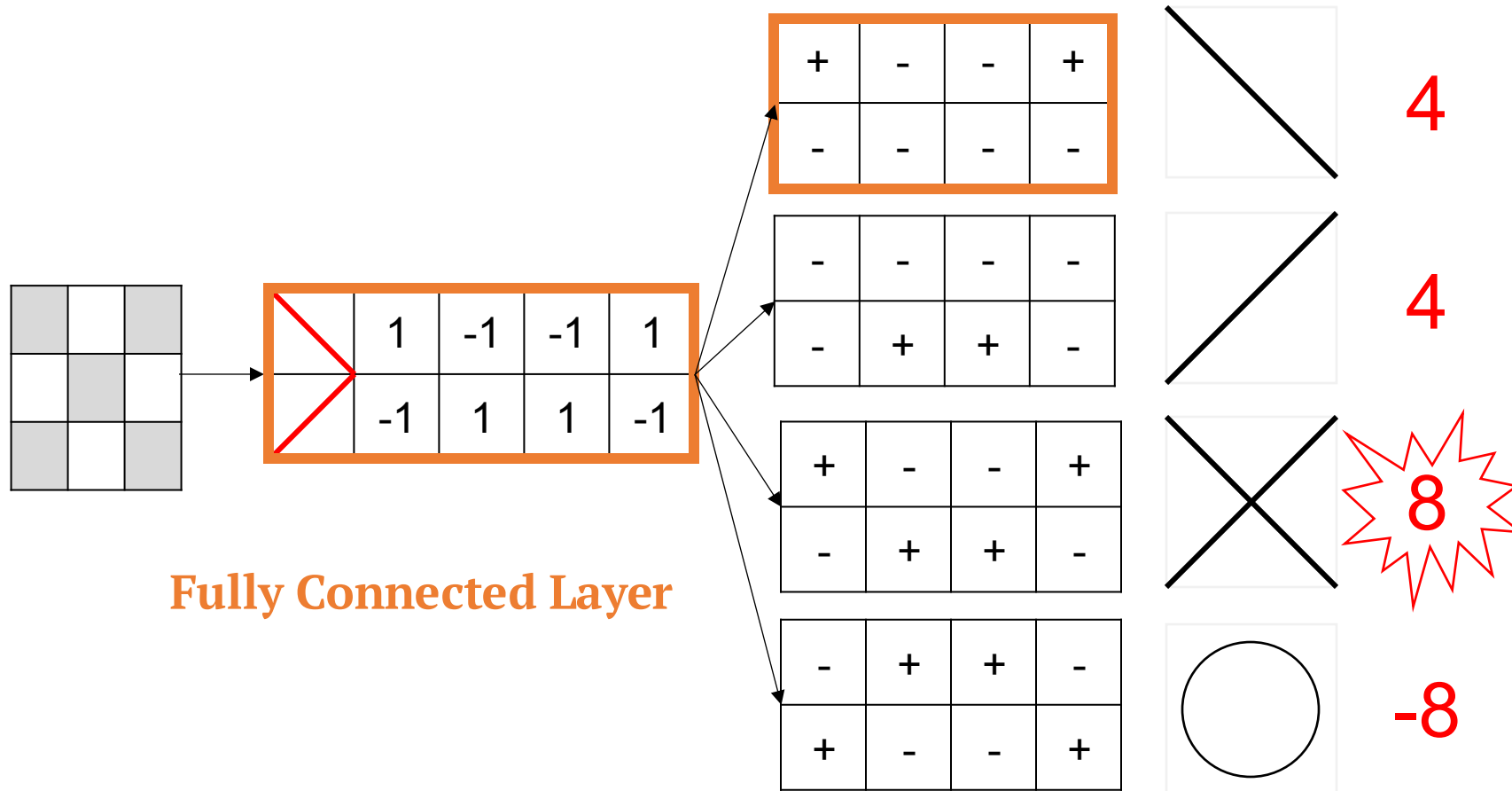


Convolutional Neural Network

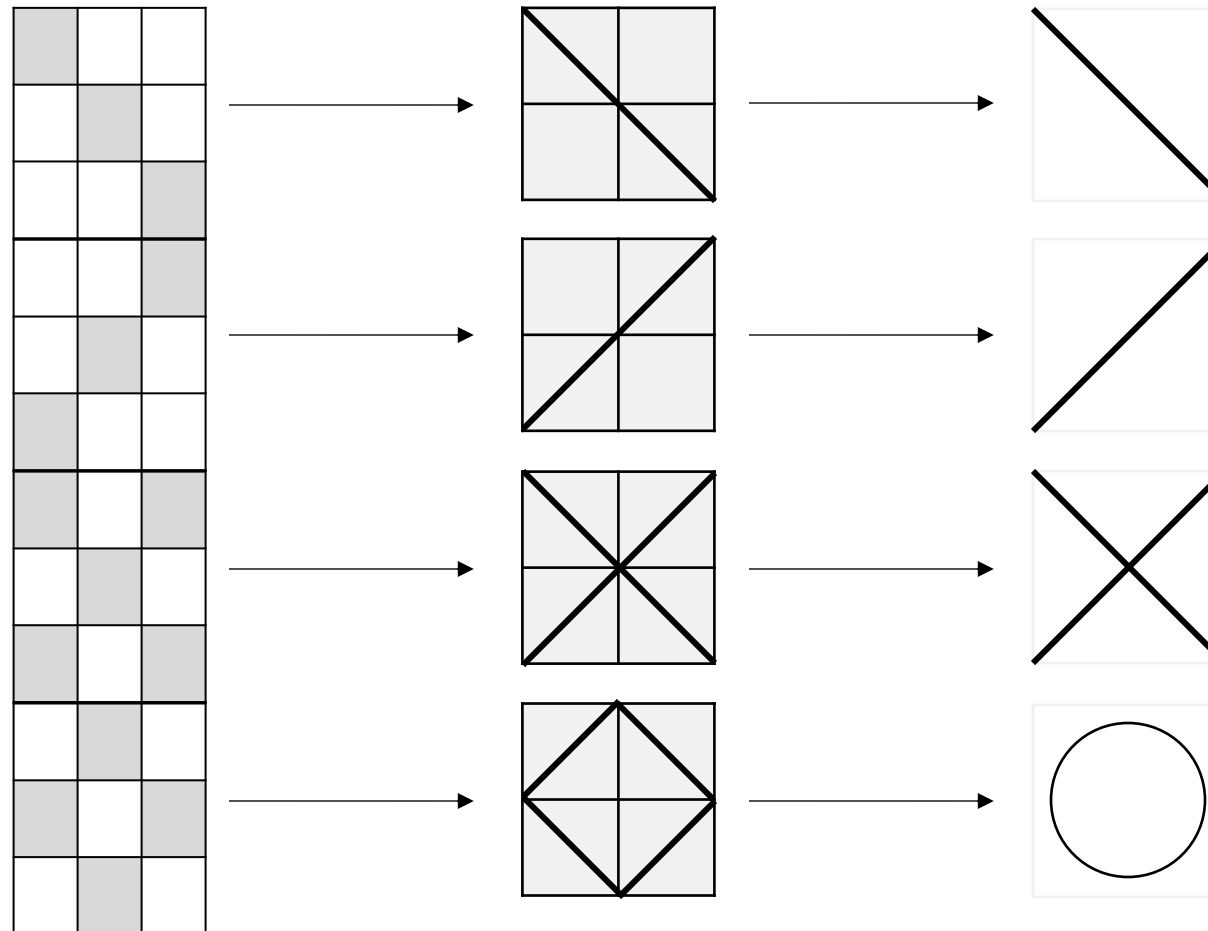
Filters



Convolutional Neural Network

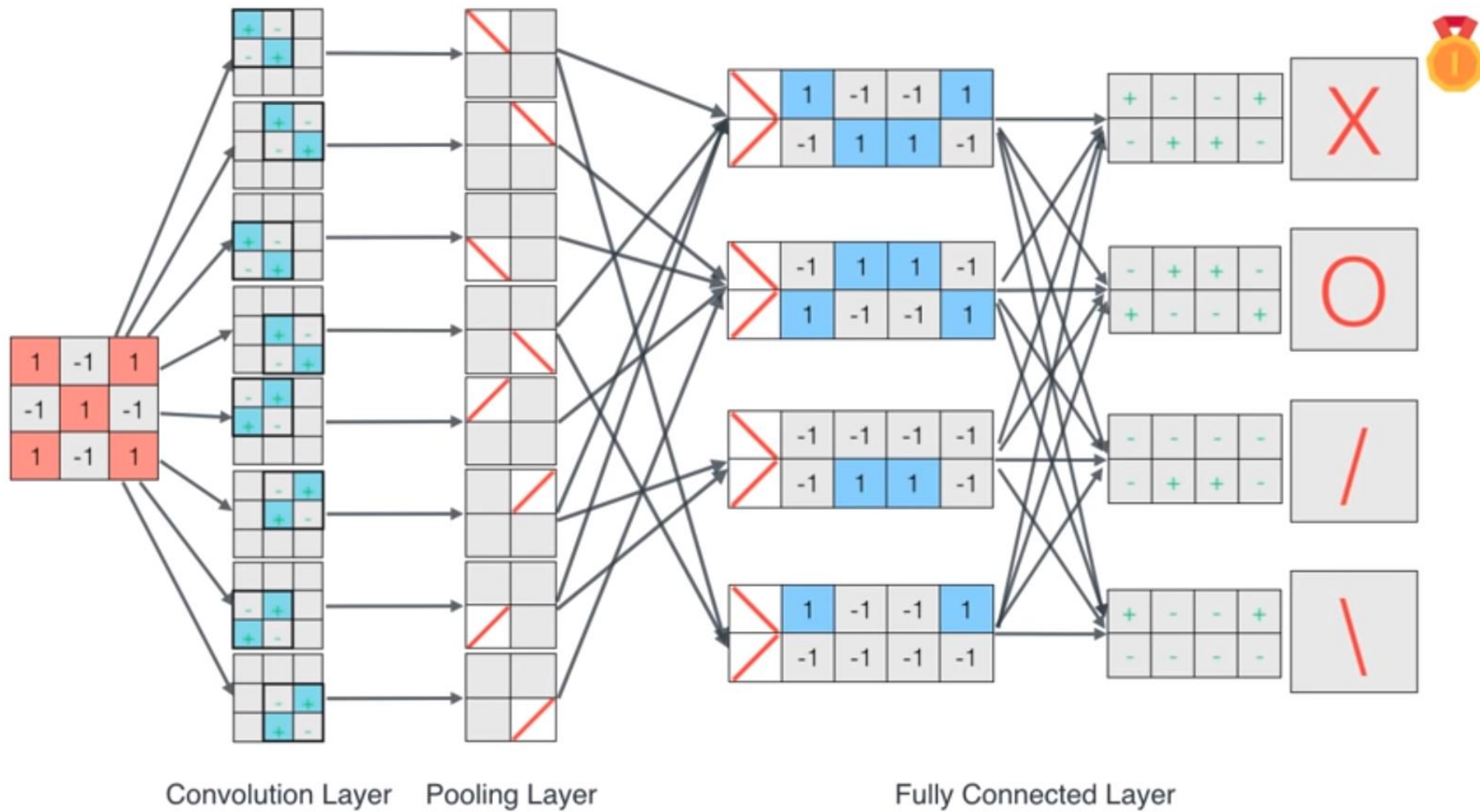


Convolutional Neural Network



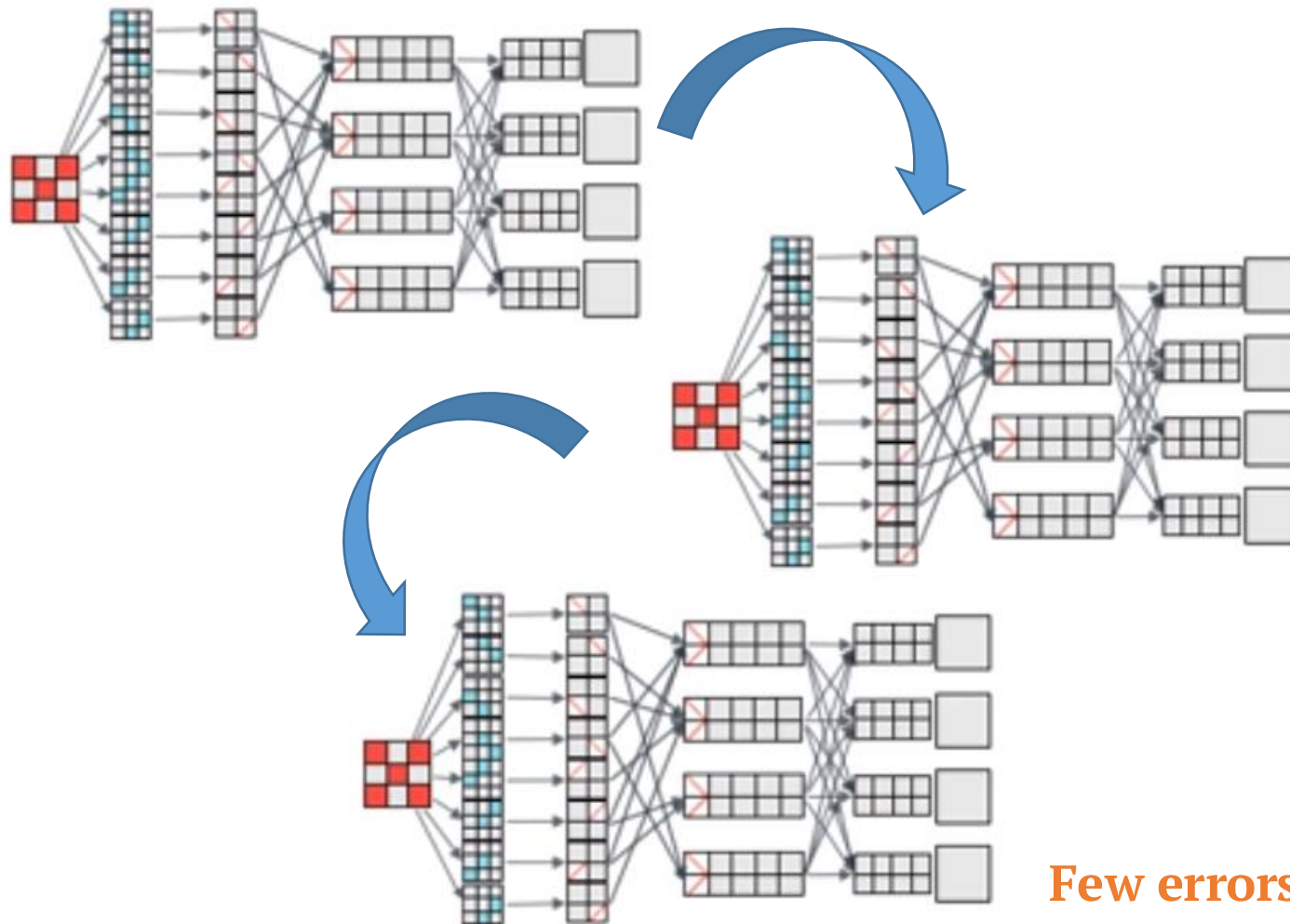
Convolutional Layer
Pooling Layer

Fully Connected Layer



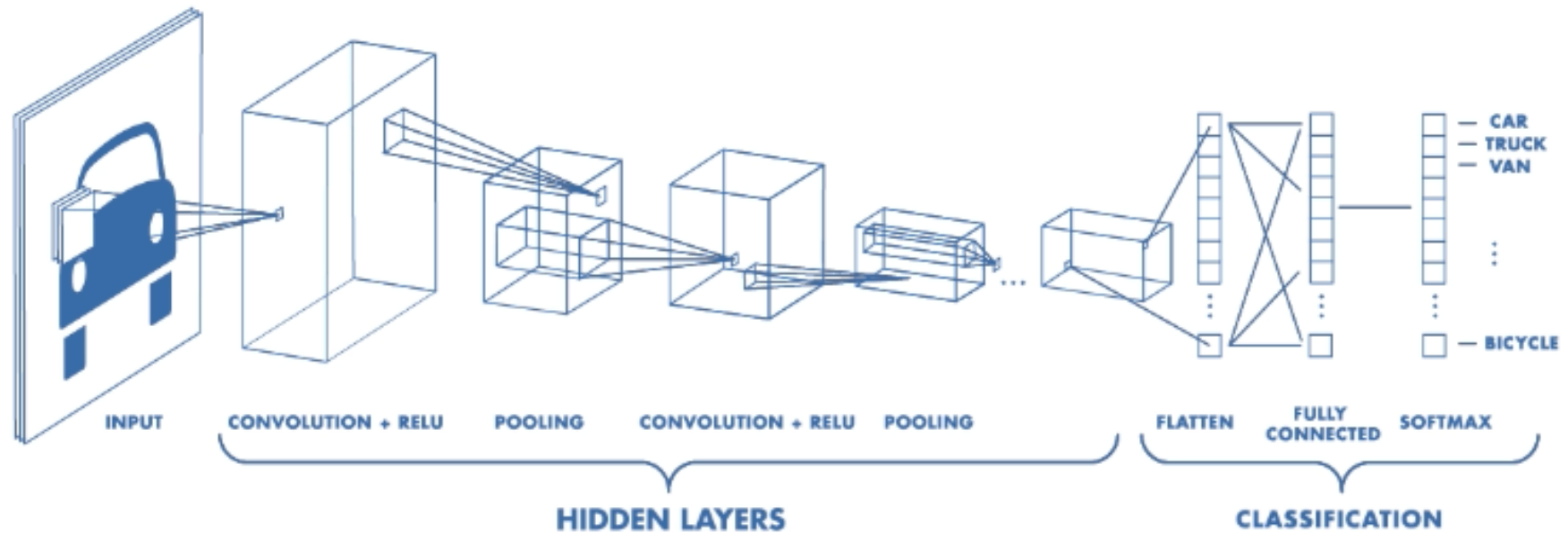
Gradient Descent

Lots of errors



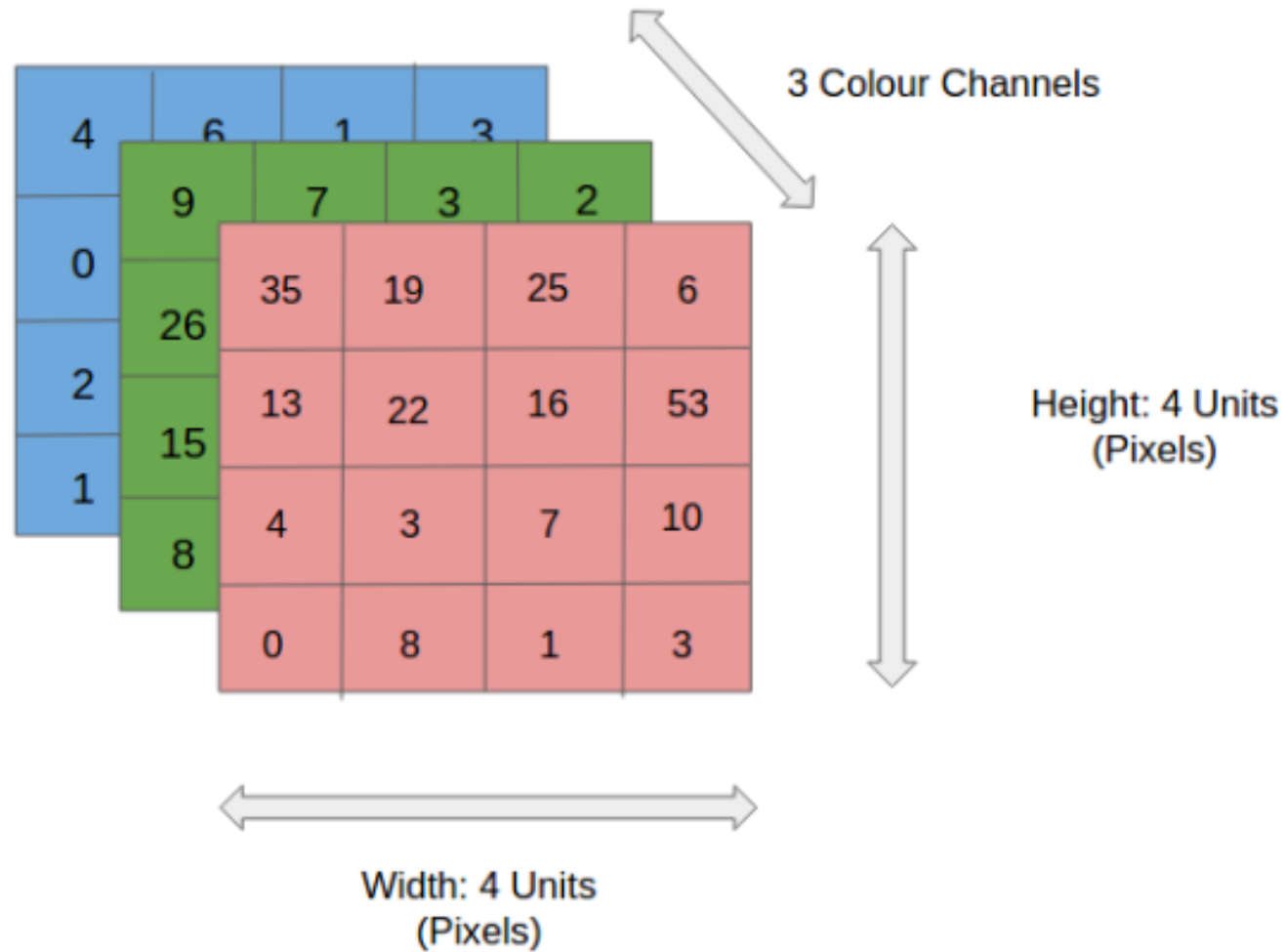
Few errors

Typical CNN architecture



Source: mathworks.com

Input Image



2 Components of CNNs

- Feature extraction – the hidden layers
 - Convolution layers – the kernel
 - Pooling layers
- Classification – the fully connected layers

Feature Extraction: Convolution Layer

1 _{x1}	1 _{x0}	1 _{x1}	0	0
0 _{x0}	1 _{x1}	1 _{x0}	1	0
0 _{x1}	0 _{x0}	1 _{x1}	1	1
0	0	1	1	0
0	1	1	0	0

Image

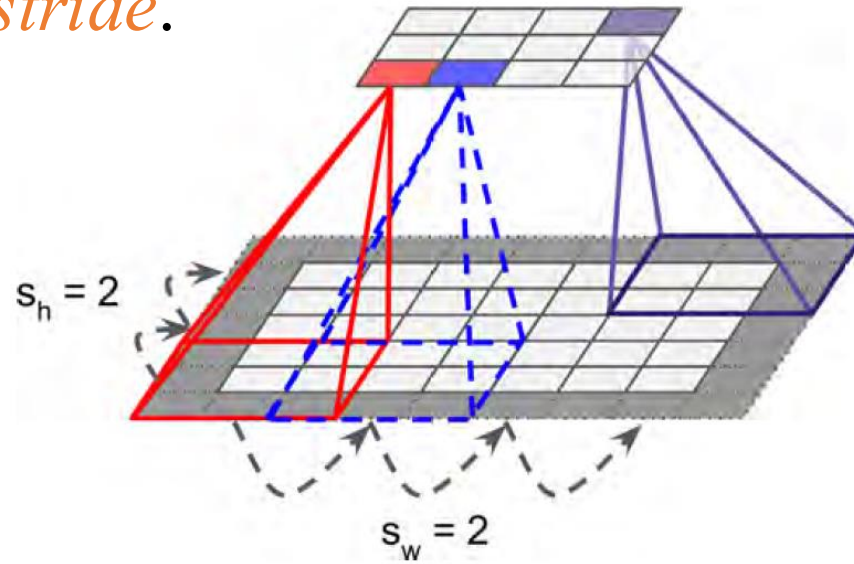
Feature Map

4		

Convolved
Feature

Convolution Layer: Strides

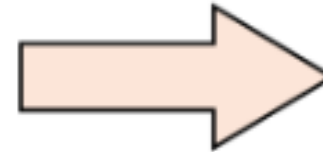
- It is also possible to connect a large input layer to a much smaller layer by spacing out the receptive fields.
- The distance between two consecutive receptive fields is called the *stride*.



Convolution Layer: Strides

1	2	3	4	5	6	7
11	12	13	14	15	16	17
21	22	23	24	25	26	27
31	32	33	34	35	36	37
41	42	43	44	45	46	47
51	52	53	54	55	56	57
61	62	63	64	65	66	67
71	72	73	74	75	76	77

Convolve with 3x3
filters filled with ones



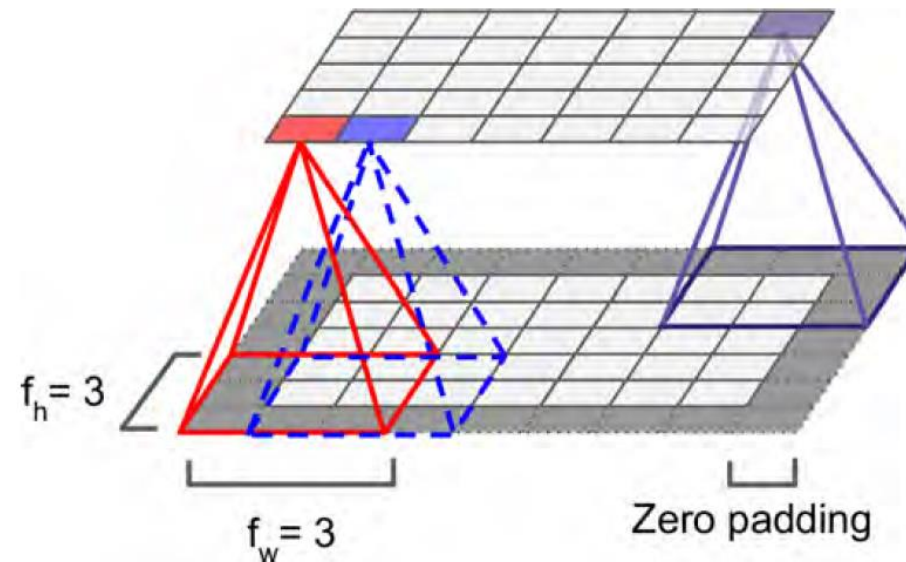
108	126	
288	306	

Stride of 2 pixels

(Source: Raghav Prabhu)

Convolution Layer: Border Effects and Padding

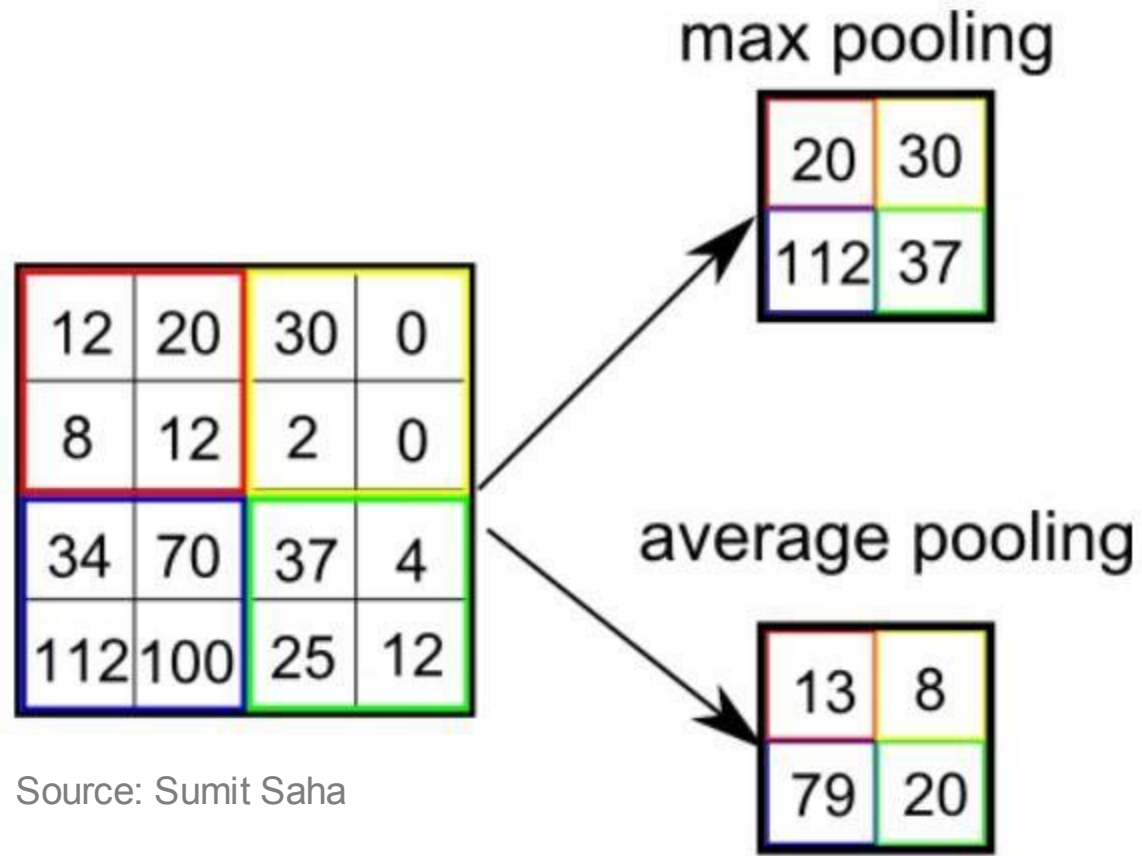
- In order for a layer to have the same height and width as the previous layer, it is common to add zeros around the inputs, as shown in the diagram. This is called *zero padding*.



Feature Extraction: Pooling Layer

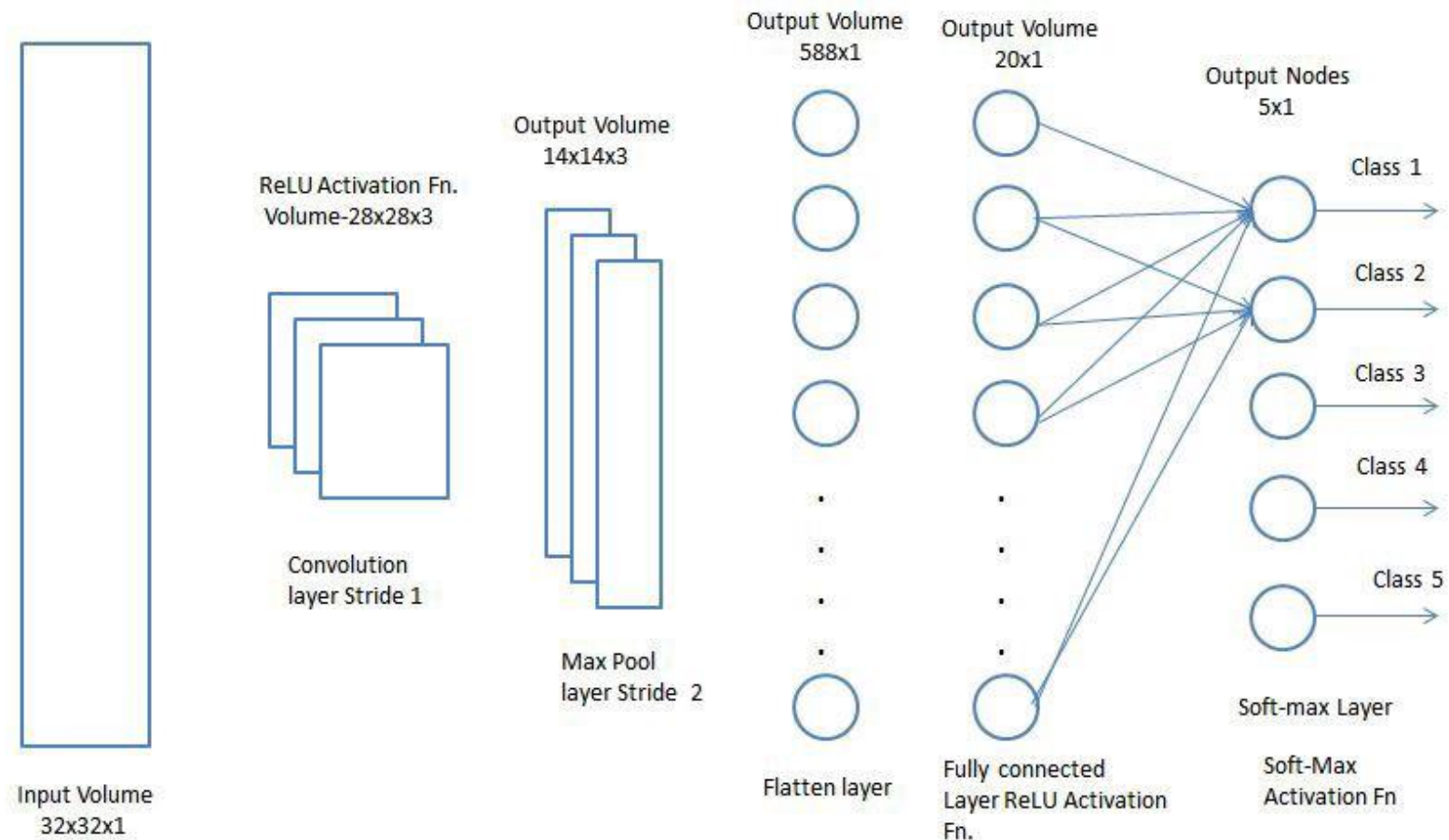
- It is common to add a pooling layer in between CNN layers
 - to continuously reduce the dimensionality
 - to reduce the number of parameters and computation in the network.
 - to shortens the training time
 - to control overfitting.
- Types of pooling
 - Max pooling
 - Average pooling
 - Sum pooling

Feature Extraction: Pooling Layer



Source: Sumit Saha

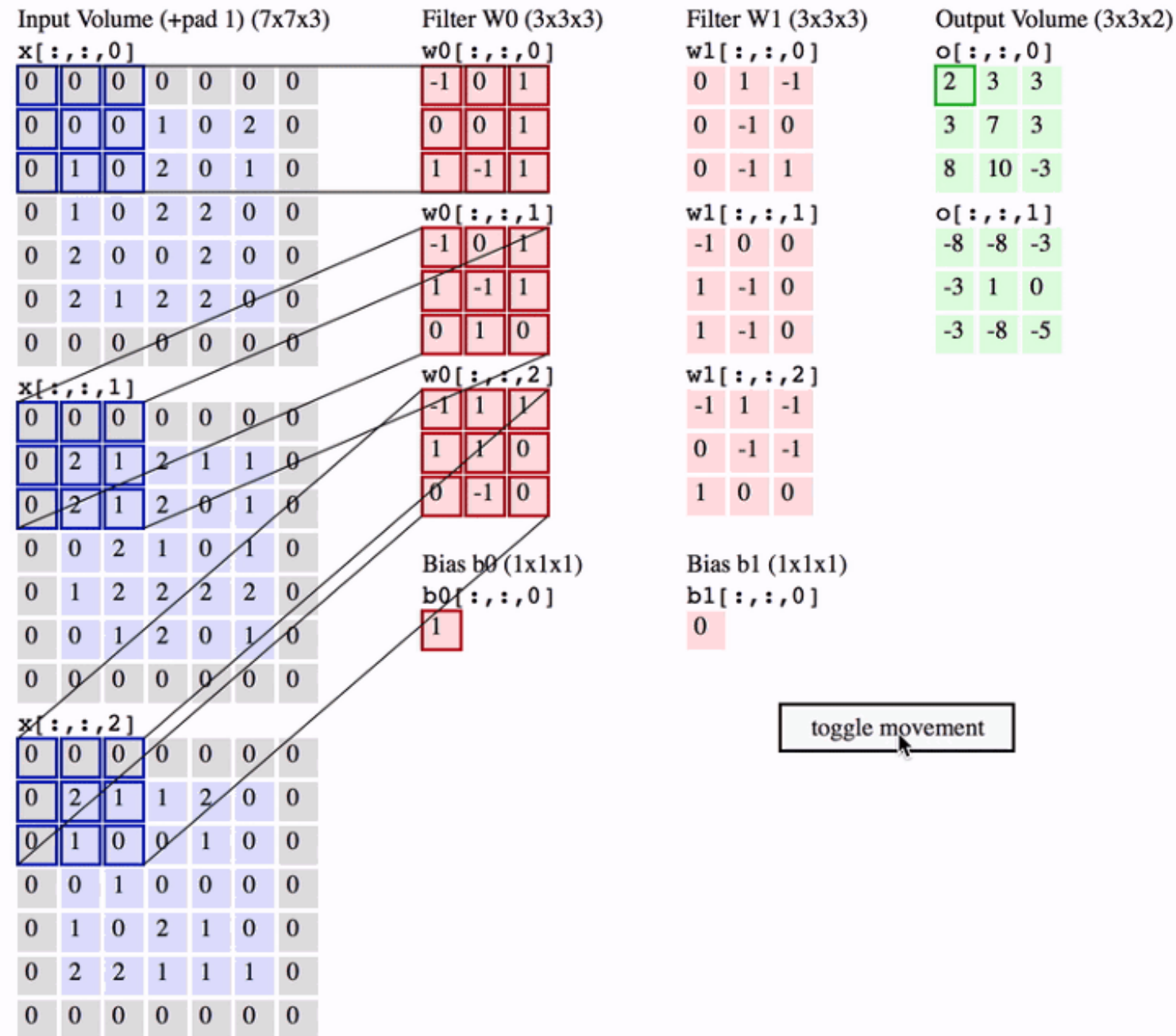
Classification: Fully Connected Layer



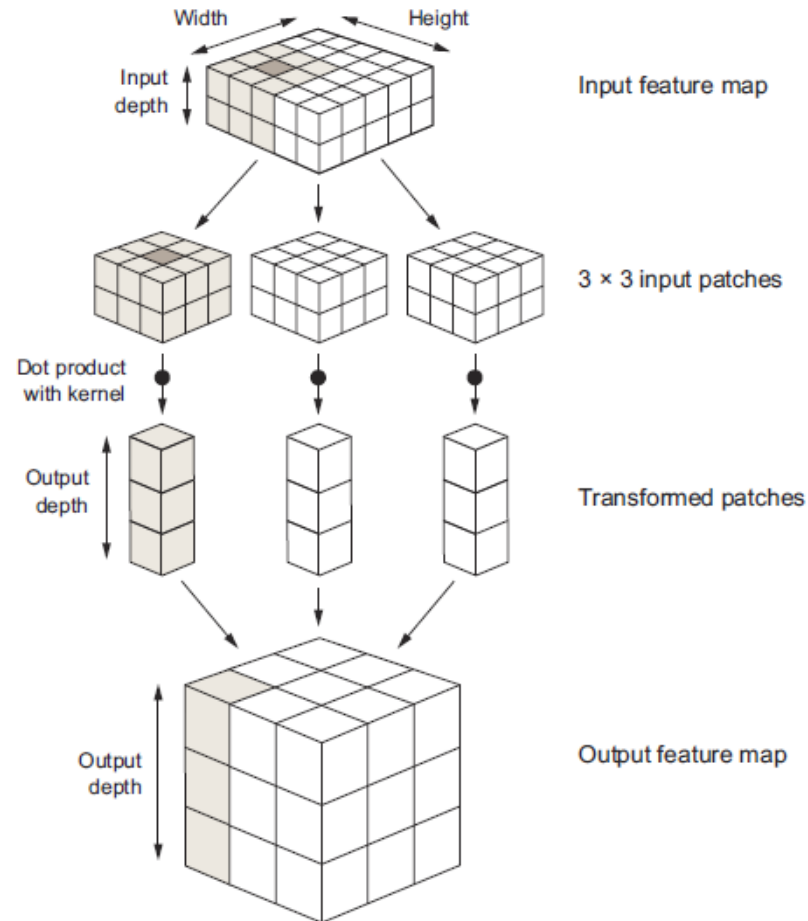
Source: Sumit Saha

CNN's 4 Key Hyperparameters

- The kernel size
- The filter count (how many filters we want to use)
- Stride (how big the steps of the filter are)
- Padding



How convolution works

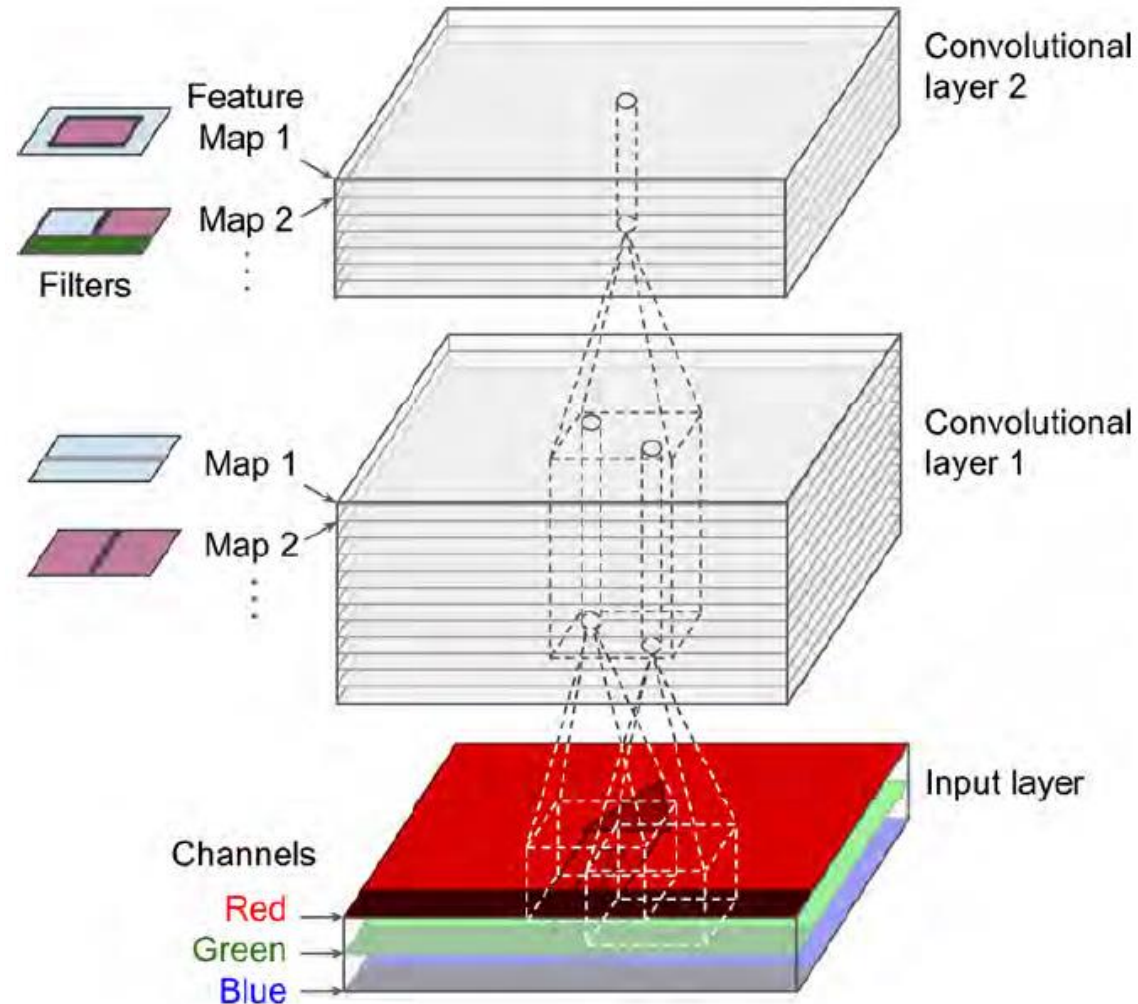


Note that the output width and height may differ from the input width and height.

They may differ for two reasons:

- **Border effects**, which can be countered by padding the input feature map
- The use of **strides**

Convolution layer with multiple feature maps



Other CNN Architectures

- Classical architecture:
 - LeNet-5 (1998)
- Three winners of the ILSVRC challenge:
 - AlexNet (2012)
 - GoogLeNet (2014)
 - ResNet (2015)

LeNet-5 Architecture

The output layer is a bit special: instead of computing the matrix multiplication of the inputs and the weight vector, *each neuron outputs the square of the Euclidian distance between its input vector and its weight vector*. Each output measures how much the image belongs to a particular digit class. The cross entropy cost function is now preferred, as it penalizes bad predictions much more, producing larger gradients and converging faster.

Layer	Type	Maps	Size	Kernel size	Stride	Activation
Out	Fully Connected	—	10	—	—	RBF
F6	Fully Connected	—	84	—	—	tanh
C5	Convolution	120	1×1	5×5	1	tanh
S4	Avg Pooling	16	5×5	2×2	2	tanh
C3	Convolution	16	10×10	5×5	1	tanh
S2	Avg Pooling	6	14×14	2×2	2	tanh
C1	Convolution	6	28×28	5×5	1	tanh
In	Input	1	32×32	—	—	—

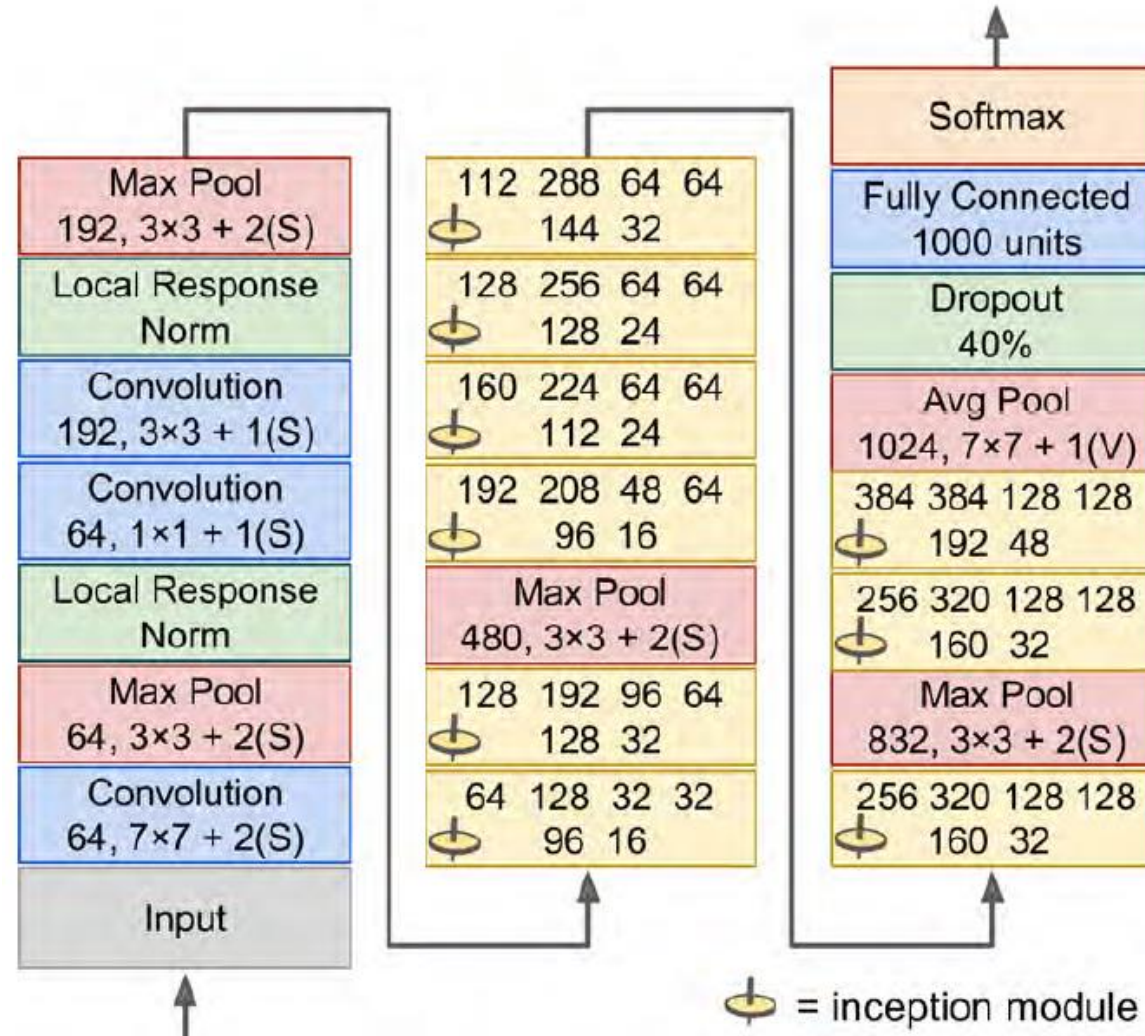
AlexNet

It is quite similar to LeNet-5, only much larger and deeper, and it was the first to *stack convolutional layers directly on top of each other*, instead of stacking a pooling layer on top of each convolutional layer.

Layer	Type	Maps	Size	Kernel size	Stride	Padding	Activation
Out	Fully Connected	—	1,000	—	—	—	Softmax
F9	Fully Connected	—	4,096	—	—	—	ReLU
F8	Fully Connected	—	4,096	—	—	—	ReLU
C7	Convolution	256	13×13	3×3	1	SAME	ReLU
C6	Convolution	384	13×13	3×3	1	SAME	ReLU
C5	Convolution	384	13×13	3×3	1	SAME	ReLU
S4	Max Pooling	256	13×13	3×3	2	VALID	—
C3	Convolution	256	27×27	5×5	1	SAME	ReLU
S2	Max Pooling	96	27×27	3×3	2	VALID	—
C1	Convolution	96	55×55	11×11	4	SAME	ReLU
In	Input	3 (RGB)	224×224	—	—	—	—

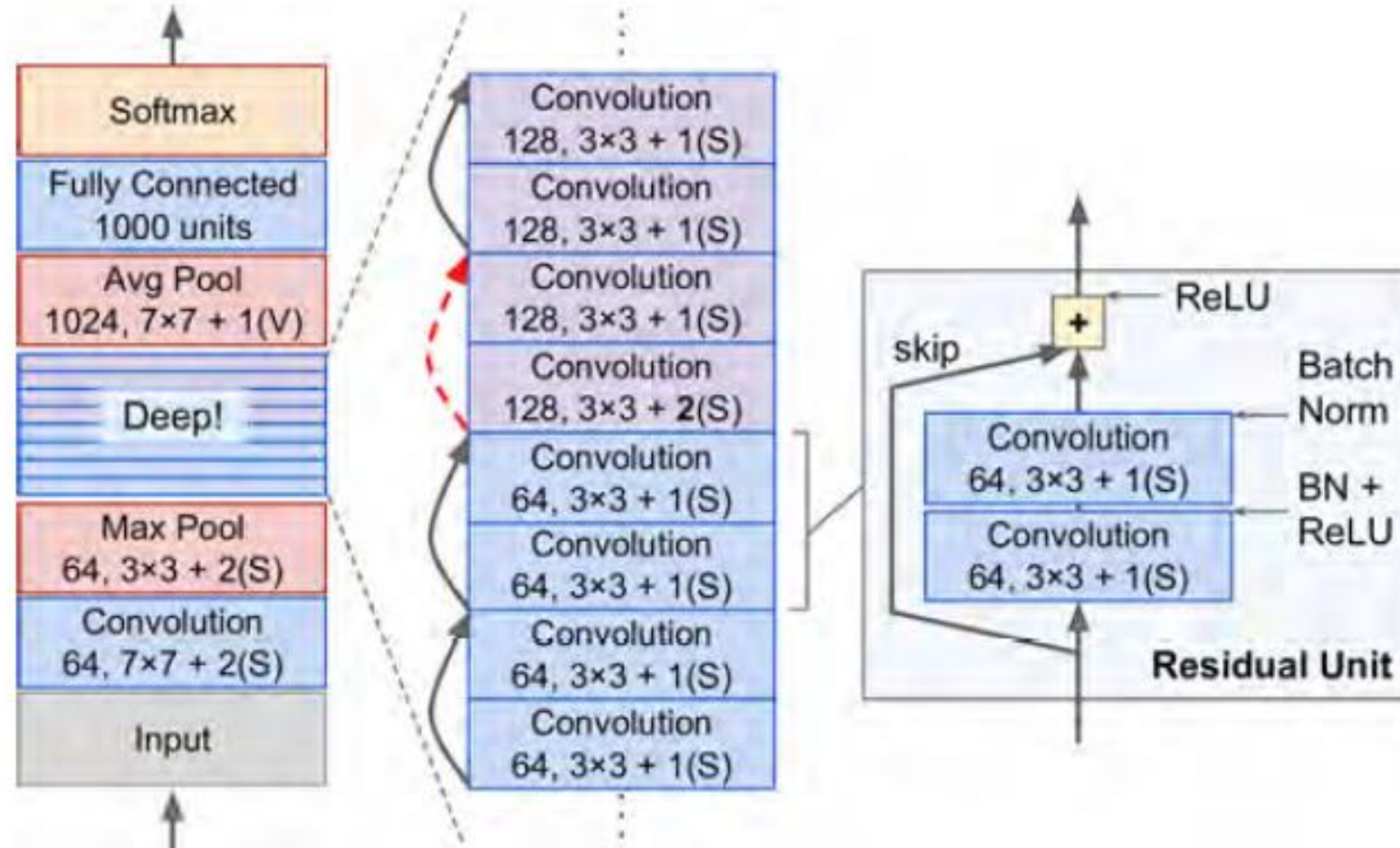
GoogLeNet

sub-networks called *inception modules*, which allow GoogLeNet to use parameters much more efficiently than previous architectures: GoogLeNet actually has 10 times fewer parameters than AlexNet (roughly 6 million instead of 60 million).



ResNet Architecture

use *skip connections* (also called *shortcut connections*):
the signal feeding into a layer is also added to the output
of a layer located a bit higher up the stack.



Transfer Learning

It is generally not a good idea to train a very large DNN from scratch: instead, you should always **try to find an existing neural network that accomplishes a similar task to the one you are trying to tackle** (we will discuss how to find them in Chapter 14), **then just reuse the lower layers of this network: this is called transfer learning**. It will not only speed up training considerably, but will also require much less training data.



"I think transfer learning is the key to general intelligence. And I think the key to doing transfer learning will be the acquisition of conceptual knowledge that is abstracted away from perceptual details of where you learned it from."

- Demis Hassabis
CEO, DeepMind

What is transfer learning?

- A machine learning technique where a model trained on one task is re-purposed on a second related task
 - For example, if you trained a simple classifier to predict whether an image contains a backpack, you could use the knowledge that the model gained during its training to recognize other objects like sunglasses.
- Mostly used in Computer Vision and Natural Language Processing Tasks
 - because of the huge amount of computational power that is needed for them

How to use transfer learning?

- Two common approaches:
 - Develop model
 - Pre-trained model

Aspect	Develop Model Approach	Pre-Trained Model Approach
Who trains the source model?	You	Someone else
Source data	You collect/train on a large related dataset	Already trained on a large public dataset
Effort	High (train from scratch)	Low (reuse existing weights)
Control	Full control over model design and training	Limited to adapting an existing model
Computation cost	High	Low
When to use	When you have big data & resources	When you have limited data & resources

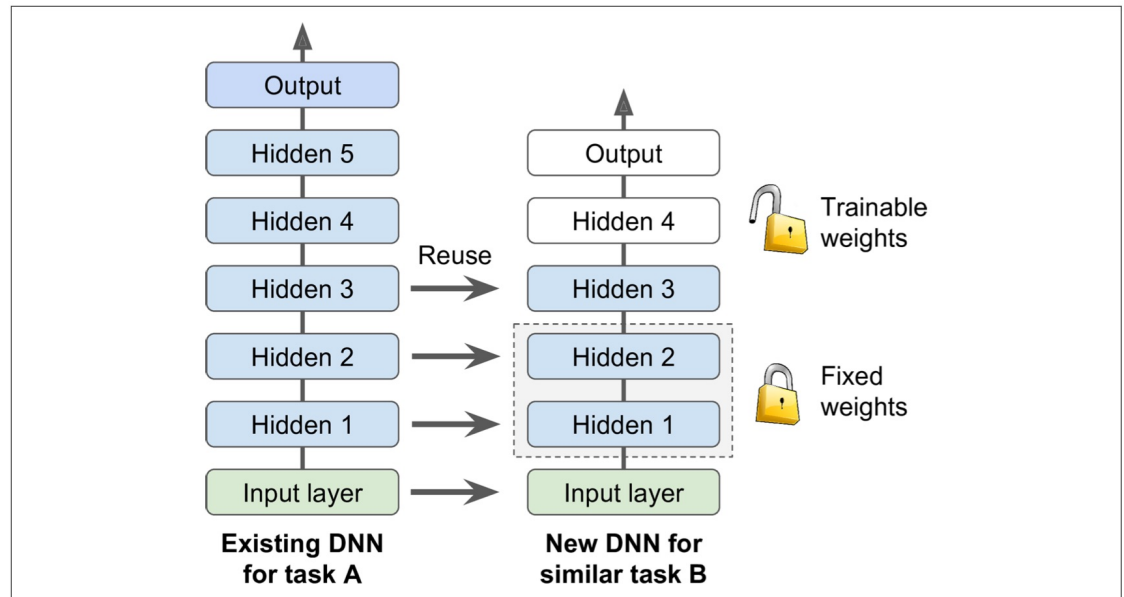


Figure 11-4. Reusing pretrained layers

Develop Model Approach

1. **Select Source Task.**

- select a related predictive modeling problem with an abundance of data

2. **Develop Source Model.**

- develop a skillful model for this first task
- The model must be better than a naive model

3. **Reuse Model.**

- The model fit on the source task can then be used as the starting point for a model on the second task of interest.
- This may involve using all or parts of the model, depending on the modeling technique used.

4. **Tune Model.**

Pre-Trained Model Approach

1. Select Source Model.

- choose an available pre-trained source model

2. Reuse Model.

- use the pre-trained model can as the starting point for the second task of interest

3. Tune Model.

* common in the field of deep learning *

Examples of Transfer Learning with Image Data

- It is common to use a deep learning model pre-trained for a large and challenging image classification task such as the **ImageNet** 1000-class photograph classification competition
- The research organizations often release their final model under a permissive license for reuse
 - **Oxford VGG Model:**
http://www.robots.ox.ac.uk/~vgg/research/very_deep/
 - **Google Inception Model**
<https://github.com/tensorflow/models/tree/master/inception>
 - **Microsoft ResNet Model**
<https://github.com/KaimingHe/deep-residual-networks>
- These models can take days or weeks to train on modern hardware.
- These models can be downloaded and incorporated directly into new models that expect image data as input.

A mostly complete chart of Neural Networks

©2016 Fjodor van Veen - asimovinstitute.org

-  Backfed Input Cell
-  Input Cell
-  Noisy Input Cell
-  Hidden Cell
-  Probabilistic Hidden Cell
-  Spiking Hidden Cell
-  Output Cell
-  Match Input Output Cell
-  Recurrent Cell
-  Memory Cell
-  Different Memory Cell
-  Kernel
-  Convolution or Pool

Perceptron (P)



Feed Forward (FF)



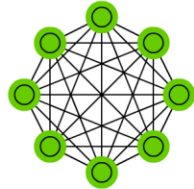
Radial Basis Network (RBF)



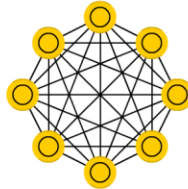
Deep Feed Forward (DFF)



Markov Chain (MC)



Hopfield Network (HN)



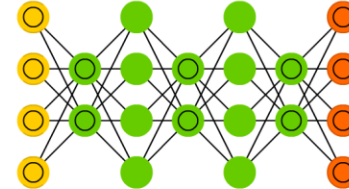
Boltzmann Machine (BM)



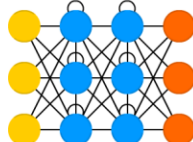
Restricted BM (RBM)



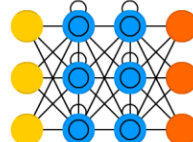
Deep Belief Network (DBN)



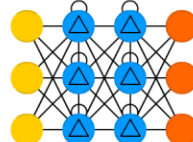
Recurrent Neural Network (RNN)



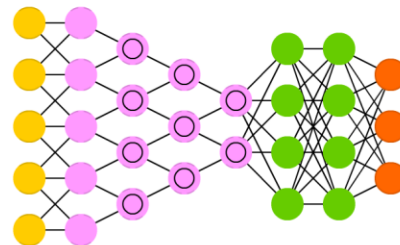
Long / Short Term Memory (LSTM)



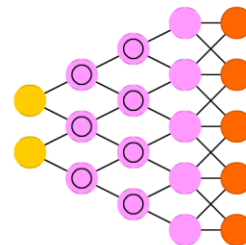
Gated Recurrent Unit (GRU)



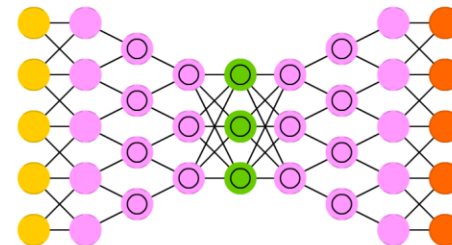
Deep Convolutional Network (DCN)



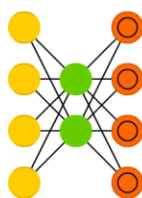
Deconvolutional Network (DN)



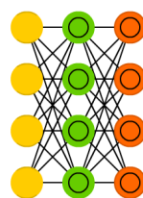
Deep Convolutional Inverse Graphics Network (DCIGN)



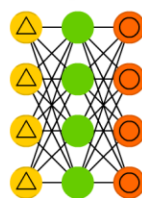
Auto Encoder (AE)



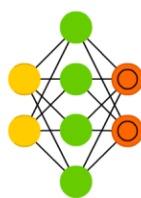
Variational AE (VAE)



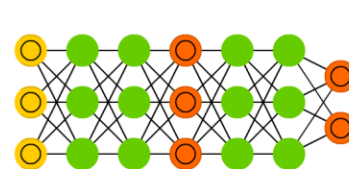
Denosing AE (DAE)



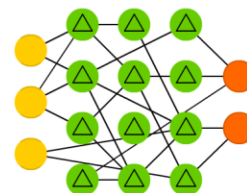
Sparse AE (SAE)



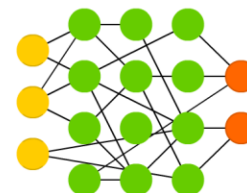
Generative Adversarial Network (GAN)



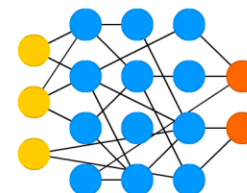
Liquid State Machine (LSM)



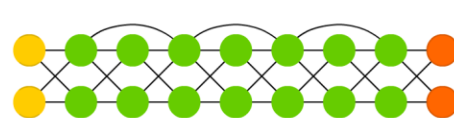
Extreme Learning Machine (ELM)



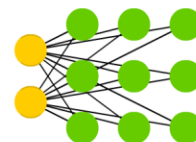
Echo State Network (ESN)



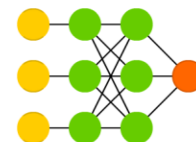
Deep Residual Network (DRN)



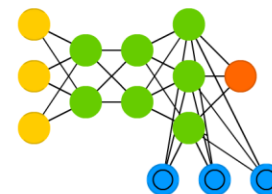
Kohonen Network (KN)



Support Vector Machine (SVM)

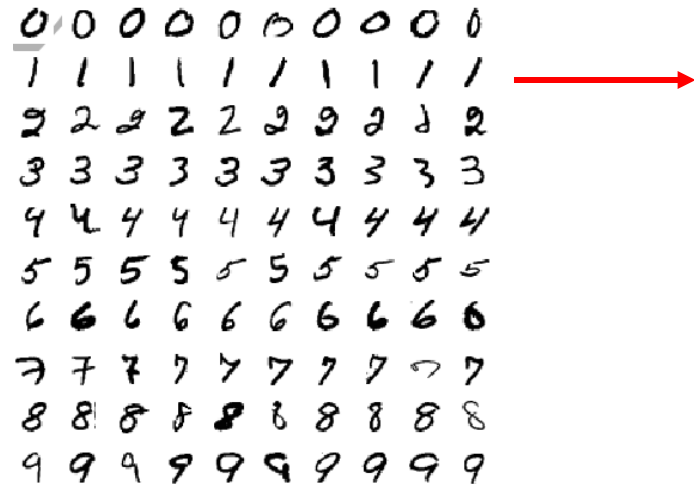


Neural Turing Machine (NTM)



Training CNN in Keras

```
cnn = models.Sequential()
cnn.add(layers.Conv2D(40, kernel_size=5, padding="same",
                      input_shape=(28, 28, 1), activation = 'relu', name = 'conv1_1'))
cnn.add(layers.MaxPool2D((2, 2)))
cnn.add(layers.Dropout(0.25))
cnn.add(layers.Conv2D(32, kernel_size=(3, 3),
                      activation='relu', kernel_initializer='he_normal', name = 'conv1_2'))
cnn.add(layers.MaxPool2D((2, 2)))
cnn.add(layers.Dropout(0.25))
cnn.add(layers.Flatten())
cnn.add(layers.Dense(64, activation='relu'))
cnn.add(layers.BatchNormalization())
cnn.add(layers.Dropout(0.25))
cnn.add(layers.Dense(10, activation='softmax'))
```



Input Mnist data

Shape (28 , 28 , 1)

```
cnn = models.Sequential()
cnn.add(layers.Conv2D(40, kernel_size=5, padding="same",
                      input_shape=(28, 28, 1), activation = 'relu',name = 'conv1_1'))
cnn.add(layers.MaxPool2D((2, 2)))
cnn.add(layers.Dropout(0.25))
cnn.add(layers.Conv2D(32, kernel_size=(3, 3),
                      activation='relu',kernel_initializer='he_normal',name = 'conv1_2'))
cnn.add(layers.MaxPool2D((2, 2)))
cnn.add(layers.Dropout(0.25))
cnn.add(layers.Flatten())
cnn.add(layers.Dense(64, activation='relu'))
cnn.add(layers.BatchNormalization())
cnn.add(layers.Dropout(0.25))
cnn.add(layers.Dense(10, activation='softmax'))
```

Covolution layer
40, 5*5

MaxPooling 2*2
Dropout(0.25)

Covolution layer
32, 3*3

MaxPooling 2*2
Dropout(0.25)

Flatten layer

Hidden layer
64 Node

BatchNormalization()
Dropout(0.25)

Output layer
10 Node

Layer (type)	Output Shape	Param #
conv1_1 (Conv2D)	(None, 28, 28, 40)	1040
max_pooling2d_1 (MaxPooling2)	(None, 14, 14, 40)	0
dropout_1 (Dropout)	(None, 14, 14, 40)	0
conv1_2 (Conv2D)	(None, 12, 12, 32)	11552
max_pooling2d_2 (MaxPooling2)	(None, 6, 6, 32)	0
dropout_2 (Dropout)	(None, 6, 6, 32)	0
flatten_1 (Flatten)	(None, 1152)	0
dense_1 (Dense)	(None, 64)	73792
batch_normalization_1 (Batch Normalization)	(None, 64)	256
dropout_3 (Dropout)	(None, 64)	0
dense_2 (Dense)	(None, 10)	650
Total params: 87,290		
Trainable params: 87,162		
Non-trainable params: 128		

Data augmentation

```
# Define a generator for train set and test set

train_datagen = image.ImageDataGenerator(rescale=1./255,
                                         rotation_range=40,
                                         width_shift_range=0.2,
                                         height_shift_range=0.2,
                                         shear_range=0.2,
                                         zoom_range=0.2,
                                         horizontal_flip=False)

test_datagen = image.ImageDataGenerator(rescale=1./255)
```

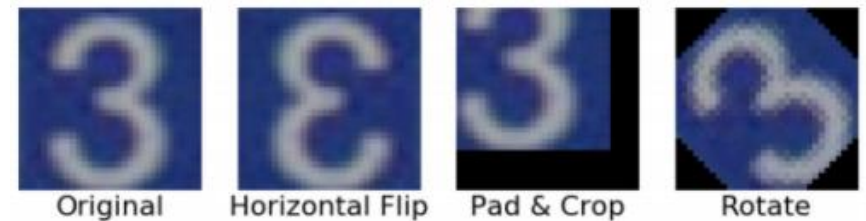
```
# Create an Iterator object.
train_generator = train_datagen.flow(X_train,y_train,
                                     batch_size = BATCH_SIZE,
                                     seed=0)

validate_generator = test_datagen.flow(X_val,y_val,
                                       batch_size = BATCH_SIZE,
                                       shuffle=False)
```

Using the ImageDataGenerator module to generate more data.

It help generate more variation of the data which help prevent overfit and generalize better.

<https://keras.io/preprocessing/image>



Transfer Learning

```
from keras.applications import vgg16

vgg = vgg16.VGG16(include_top=False,
                  weights='imagenet',
                  input_shape=(150,150,3))

prev_cnn = models.load_model('your_previos_model.h5')
prev_cnn.summary()
```

Use .pop() to remove the last layer
In this case, we want to remove last two layer

```
prev_cnn.pop()
prev_cnn.pop()
```

If we don't want to train these layer, we have to freeze these layer.

```
prev_cnn.trainable = False
```

Or Freeze a specific layers
Freeze first 3 layer

```
for i in range(3):
    prev_cnn.layers[i].trainable = False
```

What is transfer learning <https://towardsdatascience.com/transfer-learning-946518f95666>
which transfer learning method to use <https://medium.com/@14prakash/transfer-learning-using-keras-d804b2e04ef8>
The following is a tutorial code to load and freeze some layer

Evaluating a classifier

Evaluating a classifier

- Mainly evaluate using the *error rate* or *classification accuracy*.

$$\text{accuracy} = \frac{\text{Number of correct decision made}}{\text{Total number of decision made}}$$

$$\text{accuracy} = 1 - \text{error rate}$$

- The accuracy is easy to measure, but sometime not fit to real business problems.
- The accuracy has some well-known problems.
 - The importance of different errors (Unequal Costs and Benefits)
 - Unbalanced class

Confusion Matrix

- Separate out the decisions made by the classifier, making explicit how one class is being confused for another.
- Different errors can then be dealt with separately.
- A 2x2 confusion matrix

	Actual Positive (p)	Actual Negative (n)	
The model says "Yes" = positive (y)	True positives	False positives	Counts of the Errors Counts of the correct decisions
The model says "No" = not positive (n)	False negatives	True negatives	

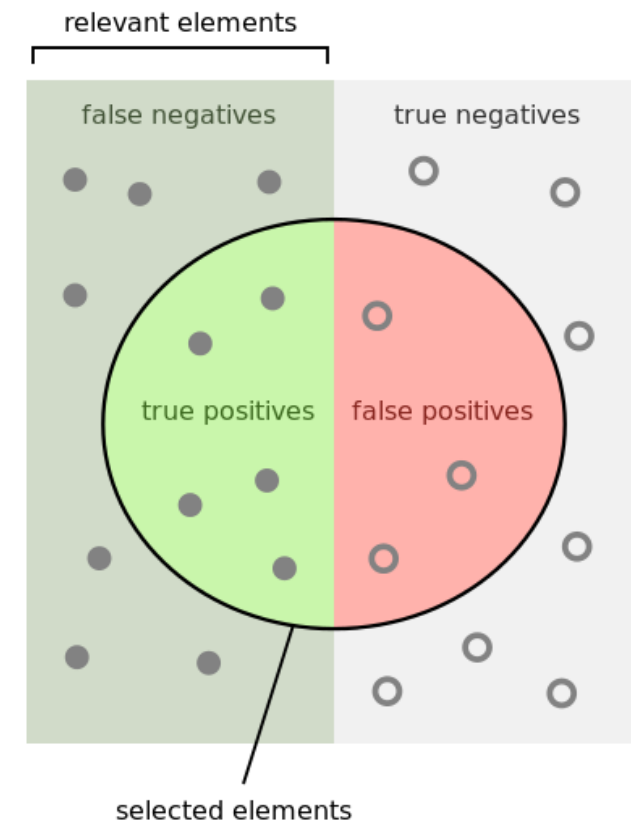
A	Churn	Not churn
Y	500	200
N	0	300

B	Churn	Not churn
Y	500	0
N	200	300

Precision and Recall

	Actual Positive (p)	Actual Negative (n)
The model says “Yes” = positive (y)	True positives	False positives
The model says “No” = not positive (n)	False negatives	True negatives

- Recall (Completeness) = true positive rate = $TP / (TP + FN)$
- Precision (Exactness) = the accuracy over the cases predicted to be positive, $TP / (TP + FP)$
- F-measure = the harmonic mean of precision and recall
= the balance between recall and precision
= $2 \cdot \frac{precision * recall}{precision + recall}$



How many selected items are relevant?

$$\text{Precision} = \frac{\text{true positives}}{\text{true positives} + \text{false positives}}$$

How many relevant items are selected?

$$\text{Recall} = \frac{\text{true positives}}{\text{true positives} + \text{false negatives}}$$

Precision and Recall

- Precision: $\frac{TP}{TP+FP}$

“Of the responses predicted YES,
how many are actually YES?”

- Recall: $\frac{TP}{TP+FN}$

“Of the responses that are actually YES,
how many are predicted YES”